MOSAIC-ST: Multimodal Integration of Spatial Transcriptomics and Whole-Slide Images for Tumor Prediction

캡스톤디자인과창업프로젝트B 2차 보고서

09 SiYa

목차

1. Team Info **1**

1.1. 과제명 2

1.2. 팀 정보 3

1.3. 팀 구성원 3

2. Project Summary **4**

2.1. 문제 정의 4

2.2. 기존 연구와의 비교 4

2.2.1. 기존 멀티모달 프레임워크 5

2.2.2. 임상 적용 측면의 한계 6

2.3. 제안 내용 6

2.4. 기대 효과 및 의의 7

2.5. 주요 기능 리스트 8

2.5.1. 멀티모달 종양 예측 프레임워크 8

2.5.2. 인코더 구조에 따른 종양 예측 성능 비교 실험 (실험 1) 9

2.5.3. 실험 1 결과 10

2.5.4. 모달리티별 종양 예측 성능과 임베딩 비교 실험 (실험 2) 11

2.5.5. 실험 2 결과 12

2.5.6. Explainability 모듈 및 case study 14

3. Project Design **17**

3.1. 요구사항 정의 17

3.1.1. 기능 요구사항 17

3.1.2. 비기능 요구사항 17

3.1.3. 요구사항 명세 17

3.2. 전체 시스템 구성 18

3.2.1. 데이터셋 구성 18

3.2.2. 학습 설정 19

3.2.3. 실험 설정 19

3.3. 주요 엔진 및 기능 설계 20

3.4. 주요 기능의 구현 21

3.4.1. Fusion 모듈 설계 및 실험 구현 20

3.4.2. Modality 실험 구현 27

3.4.3. XAI 모듈 구현 31

4. Appendix **38**

1. Team Info

1.1. 과제명

MOSAIC-ST: Multimodal Integration of Spatial Transcriptomics and Whole-Slide Images for Tumor Prediction

1.2. 팀 정보

09조 SiYa

1.3. 팀 구성원

- 2303050 박은서 (팀장)
- 2271040 유지혜
- 2376087 류다현

2. Project Summary

2.1. 문제 정의

병리학자는 암 진단을 위해 전체 슬라이드 이미지(Whole-SIide Images, WSIs)에 크게 의존한다. 그러나 WSIs는 종양 진행의 근본적인 분자적 메커니즘에 대한 정보를 충분히 제공하지 못한다. 한편, 공간 전사체학(Spatial Transcriptomics, ST)은 공간적으로 정렬된 유전자 발현 정보를 제공하여 조직학적 이미지의 한계를 보완할 수 있지만, 높은 비용과 긴 처리 시간으로 인해 일상적인 임상 적용에는 제약이 존재한다. 이러한 한계는 형태학적 정보와 분자적 정보를 동시에 활용하여 종양을 예측할 수 있는 계산 모델의 필요성을 제기한다.

최근 ST와 WSI를 통합하려는 연구가 진행되고 있으나, 많은 기존 방법들은 late fusion 또는 단순 feature concatenation에 의존하고 있어, 두 모달리티 간의 의미 있는 상호작용을 충분히 포착하지 못하는 한계가 있다. 또한 ST 데이터는 종종 단순한 수치 벡터로 처리되어 유전자 간의 의미적 관계를 반영하지 못하며, 이로 인해 형태학적 정보와 분자적 정보 간의 관계를 해석하는 데에도 제한이 있다.

더불어, 최근 Medical AI 분야에서는 WSI와 유전자 발현 데이터 간의 상호보완적 정보를 활용하기 위한 다양한 연구가 진행되어 왔다. 특히 ST 데이터를 포함한 멀티모달 학습은 암 진단과 같은 과제에서 단일 모달리티 대비 우수한 성능을 보이는 것으로 보고되었다. 그러나 기존 연구들은 구조적 설계 및 평가 관점에서 설명 가능성(explainability)과 범용성(generalizability)이라는 공통적인 한계를 가지며, 이는 실제 임상 적용 가능성을 저해하는 요소로 작용한다.

2.2. 기존 연구와의 비교

기존의 단일 모달리티 기반 접근 방식은 각각의 정보에 편향된 한계를 가진다. 이미지 기반 모델은 국소적인 형태학적 패턴에 집중하는 반면, ST 기반 모델은 공간적 구조를 충분히 반영하지 못한 채 전사체 차이에 의존하는 경향이 있다. 이로 인해 각 모델의 설명이 단편적이며 특정 모달리티에 치우치는 문제가 발생한다.

또, 기존 연구들은 주로 멀티모달 결합 자체의 효과를 보이거나 특정 구조를 제안하는 데 초점을 맞추었으며, ST를 멀티모달 입력으로 사용할 때 "어떤 방식으로 활용하는 것이 효과적인지"에 대한 체계적인 비교 분석은 충분히 이루어지지 않았다. 또한, 멀티모달 구성 내에서 ST가 가지는 고유한 기여도를 다른 생물학적 모달리티와 비교하여 분석한 연구 역시 제한적이다.

반면, 본 연구에서 제안한 멀티모달 통합 방식은 임베딩 구조, 공간적 attention, 그리고 유전자 중요도 패턴 간의 정합성을 확보함으로써 보다 일관된 표현을 학습한다. 특히, 중요도가 높은 영역은 조

직 좌표와 임베딩 공간에서 일관되게 나타나며, 이에 대응하는 유전자들은 면역 반응, 혈관 신생, 기질 재구성과 같은 생물학적 과정과 연관되는 것으로 확인되었다. 이는 ST 데이터가 형태학적 정보와 분자적 정보를 연결하는 효과적인 매개체로 작용하며, 구조적으로 정렬된 설명을 가능하게 함으로써 공간 기반 종양 예측의 해석 가능성을 크게 향상시킴을 시사한다.

이를 바탕으로, 본 연구는 다양한 실험 설정에서의 성능 비교를 통해 ST를 멀티모달 환경에서 효과적으로 활용하는 방안을 제시하고, 임베딩 추출 및 시각화를 통해 모델 내부 표현을 분석함으로써 ST의 좌표 정보가 발현 데이터와 이미지 정보 간의 상호작용을 어떻게 매개하는지를 정량적·정성적으로 검증한다.

2.2.1. 기존 멀티모달 프레임워크

초기 멀티모달 접근 방식인 Pathomic Fusion[1]은 WSI에서 추출한 CNN 기반 feature와 유전체 데이터를 단순 결합(concatenation)하여 예후를 예측하는 구조를 제안하였다. 이 모델은 멀티모달 통합이 단일 모달리티 대비 성능 향상에 기여할 수 있음을 실험적으로 입증했다는 점에서 중요한 의미를 가진다. 그러나 두 모달리티를 독립적인 feature 공간으로 간주하고 단순 결합하는 방식은 형태학적 특징과 분자적 정보 간의 구조적 대응 관계를 학습하지 못한다는 한계를 가진다. 예를 들어, 특정 종양의 stromal morphology가 면역 반응 관련 유전자 발현과 연관된다는 사실을 모델이 명시적으로 학습하거나 해석할 수 없다. 즉, Pathomic Fusion은 멀티모달 학습의 출발점으로서 의미는 있으나, 모달리티 간 상호작용을 고려하지 않는다는 점에서 표현력과 해석 가능성 모두 제한적이다. 또한 이러한 초기 멀티모달 접근들은 특정 코호트나 한정된 암종에서만 검증되는 경우가 많아, 범용성 검증이 부족하다.

이후 등장한 BrST-Net[2]은 WSI로부터 유전자 발현을 직접 예측하는 방향으로 연구를 확장하였다. 이 모델은 유방암 병리 영상으로부터 유전자 발현을 직접 회귀하는 CNN 기반 모델로, 이미지 기반으로 transcriptomic 정보를 추정할 수 있으며 morphology만으로도 일정 수준의 molecular signal을 복원할 수 있다는 가능성을 제시하였다. 그러나 BrST-Net은 주로 gene-level regression에 초점을 맞추고 있으며, 실제 임상에서 중요한 종양 진단 또는 예후 예측과 같은 downstream task와의 연결이 제한적이다. 또한 유전자 간의 상호작용이나 기능적 맥락을 고려하지 않고 각 유전자를 독립적인 회귀 대상으로 취급하기 때문에, 생물학적 해석 가능성을 제한하는 요인으로

---

[1] Chen RJ et al. Pathomic Fusion: An Integrated Framework for Fusing Histopathology and Genomic Features for Cancer Diagnosis and Prognosis. IEEE Trans Med Imaging. 2022;41(4):757-770.

[2] Rahaman MM et al. Breast cancer histopathology image-based gene expression prediction using spatial transcriptomics data and deep learning. Sci Rep. 2023;13(1):13604.

작용한다.

보다 최근에는 GIST[3]와 같은 모델이 제안되며 multimodal fusion 구조가 한층 발전하였다. 이 계열의 연구에서는 대규모 histopathology 데이터로 사전 학습된 vision encoder와 graph 또는 transformer 기반 구조를 결합하여, ST 데이터를 활용한 공간적transcriptomic 정보를 반영하고자 하였다. 특히 graph-based representation을 통해 spatial adjacency를 고려하거나, attention mechanism을 통해 일부 장거리 상호작용을 반영하려는 시도가 이루어졌다. 그러나 이러한 접근 역시 ST 데이터를 주로 수치 벡터 또는 그래프 구조로 표현함으로써, 유전자 간의 semantic 관계나 co-expression 패턴을 충분히 반영하지 못한다는 한계를 가진다. 또한 복잡한 모델 구조에도 불구하고, 실제로 어떤 유전자와 조직 패턴이 결합되어 예측에 기여하는지에 대한 해석 가능성은 여전히 제한적인 수준에 머물러 있다.

2.2.2. 임상 적용 측면의 한계

기존 연구의 중요한 문제 중 하나는 explainability의 부족이다. 대부분의 모델은 예측 정확도 향상에 초점을 맞추고 있으며, 모델이 어떤 근거로 특정 예측을 수행했는지를 명확히 제시하지 못한다. 이는 질병의 원인 규명과 진단 근거 제시가 중요한 의료 분야에서 치명적인 약점으로 작용한다. 예를 들어, attention 기반 MIL 구조는 일부 조직 영역의 중요도를 시각화할 수 있으나, 해당 영역이 어떤 분자적 신호와 연결되는지는 설명하지 못한다. 반대로 transcriptomics 기반 모델은 중요한 유전자를 도출할 수 있으나, 그것이 실제 조직 내 어떤 형태학적 패턴과 대응되는지는 파악하기 어렵다. 즉, 기존 모델들은 모달리티 내부의 해석은 일부 가능하지만, 모달리티 간 연결 관계에 대한 해석은 거의 제공하지 못한다.

또한 모델의 범용성(generalizability) 역시 중요한 한계로 지적된다. 많은 연구들이 특정 데이터셋에 최적화되어 있으며, 데이터셋이 변경될 경우 성능이 크게 저하되는 문제가 보고되고 있다. 이는 모델이 데이터셋 특이적 feature에 overfitting되었음을 의미하며, 실제 임상 환경에서 다양한 환자군과 병원 데이터를 다뤄야 하는 상황에서는 치명적인 제약이 된다

2.3. 제안 내용

기존 연구들을 종합하면 다음과 같은 공통적인 한계가 도출된다. 첫째, 기존 연구들은 explainability 측면에서 한계를 가지며, 모델의 의사결정 근거를 명확히 제시하지 못한다. 이는 질병의 원인 규명과

---

[3] Ge Y et al. Deep learning-enabled integration of histology and transcriptomics for tissue spatial profiling (GIST). Research. 2025.

진단 근거를 명확히 하는 것이 중요한 의료 분야에서, 모델 결과를 임상의가 신뢰하고 활용하기 어렵게 한다. 둘째, 특정 데이터셋에 의존하는 경향이 강하여 다양한 환경에서의 일반화 성능이 충분히 검증되지 않았다. 셋째, ST 데이터를 효과적으로 활용하고 해석하는 연구는 아직 초기 단계에 머물러 있어, 이를 활용한 멀티모달 모델임에도 불구하고 모달리티 간 상호작용을 충분히 반영하지 못하고 단순 feature 결합 수준에 머무르는 경우가 많다.

따라서 본 연구에서는 기존 연구들의 한계를 극복하기 위해, ST와 WSI 간 상호작용을 명시적으로 모델링하는 이중 인코더 기반 멀티모달 프레임워크 **MOSAIC-ST(Multi-mOdal-Spatial AI for Cross-modal learning)**를 제안한다. ST 데이터는 BERT 기반의 인코더를 통해 유전자 토큰 시퀀스로 인코딩, WSI는 ResNet 기반 인코더를 통해 처리한 후 cross-modal attention을 통해 형태학적 특징과 유전자 간의 대응 관계를 학습한다. 이때 다양한 구조의 모델 옵션에 대해 예측 정확도를 비교하고, 모델이 생성한 representation을 분석하여 학습 양상을 파악함으로써 ST 기반의 멀티모달 학습에서 핵심적인 기능을 하는 요소가 무엇인지 규명한다. 더불어 explainability를 핵심 요소로 고려하여 주요 유전자, 형태학적 패턴, 공간 정보를 정량적으로 제시하고, HEST-1K[4] 및 STimage-1K4M[5]과 같은 서로 다른 데이터셋을 활용하여 모델의 범용성을 검증함으로써 기존 연구 대비 실질적인 임상 적용 가능성을 갖는다.

2.4. 기대 효과 및 의의

본 연구는 데이터 규모가 제한된 환경에서도 ST 기반 멀티모달 모델이 성능, 해석 가능성, 데이터 효율성 간의 균형을 효과적으로 달성할 수 있음을 보여준다. 이는 실제 임상 환경과 같이 데이터 확보가 어려운 상황에서도 적용 가능한 실용적인 접근법을 제시한다는 점에서 중요한 의의를 가진다.

한편, 본 연구는 ST–WSI 쌍 데이터의 규모가 제한적이며, 설명 가능성 분석이 주로 정성적 평가에 기반한다는 한계를 가진다. 또한, 대규모 파운데이션 모델이나 보다 복잡한 크로스모달 구조와의 비교는 수행하지 못하였으며, 이는 향후 더 큰 데이터셋과 정교한 벤치마크를 통해 보완될 필요가 있다.

그럼에도 불구하고, 본 연구는 90% 이상의 종양 예측 정확도와 동시에 설명 가능성, 범용성을 확보하여 저자원 환경에서 ST와 WSI를 효과적으로 통합한 멀티모달 학습의 가능성을 제시하며, 향후 공간 전사체 기반 종양 예측 및 해석 가능한 의료 인공지능 연구에 중요한 기초를 제공한다.

---

[4] Jaume G et al. HEST-1k: A Dataset for Spatial Transcriptomics and Histology Image Analysis. arXiv. 2024.

[5] Chen J et al. STimage-1K4M: A histopathology image-gene expression dataset. arXiv preprint arXiv:2406.06393. 2024.

2.5. 주요 기능 리스트

2.5.1. 멀티모달 종양 예측 프레임워크

본 연구에서는 spatial transcriptomics(ST)와 whole slide image(WSI)를 결합한 멀티모달 종양 예측 프레임워크를 제안한다. 제안하는 프레임워크는 ST와 이미지 정보를 통합적으로 활용하여 종양 예측 성능을 향상시키는 것을 목표로 하며, 프레임워크의 전체 구조는 Figure 1에 제시되어 있다.

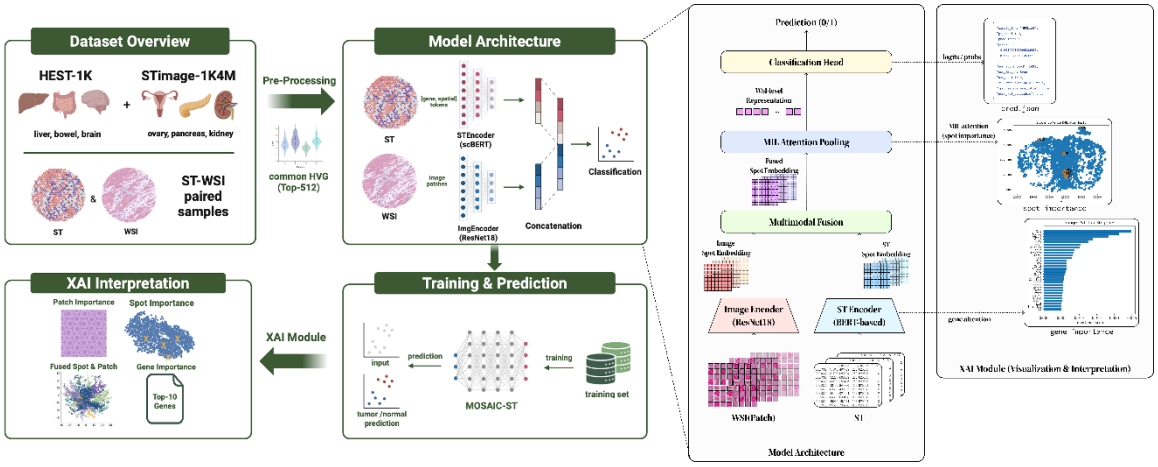


Figure 1. MOSAIC-ST의 전체 파이프라인(좌측)과 모델 구조 overview(우측). 데이터셋 → 전처리 → 모델 입력 → 모델 학습 → 설명가능성 확보의 단계를 거침.

WSI-ST 쌍의 샘플들로 이루어진 HEST-1K, STimage-1K4M 데이터셋을 확보하고 전처리해 모델 입력으로 사용하며, 학습한 모델로 inference를 수행한다. Inference 결과 종양 여부뿐 아니라 모델이 의사 결정에 중요하게 고려한 spot과 유전자를 출력해 explainability를 확보한다. 데이터셋과 실험 설정에 대한 자세한 서술은 3.2.에 제시한다.

모델은 WSI와 ST 데이터를 별도의 인코더로 전달받으며, 각 인코더에서는 spot(patch) 단위 representation이 추출되고 이는 fusion layer에서 spot-level representation으로 통합된다. 이후 sample-level 결과를 생성하기 위해 MIL(Multi Instance Learning) attention pooling을 수행하며, classification head에 pooling된 sample-level representation을 통과시켜 binary 예측 결과(Healthy(0)/Cancer(1))을 생성한다. 이때 각 모듈에서는 모델의 의사결정 근거를 분석할 수 있는 gene attention score, MIL attention score, logits/probs가 단계별로 추출되며, XAI module에서 이들 값을 시각화해 설명가능성을 확보하도록 구현한다.

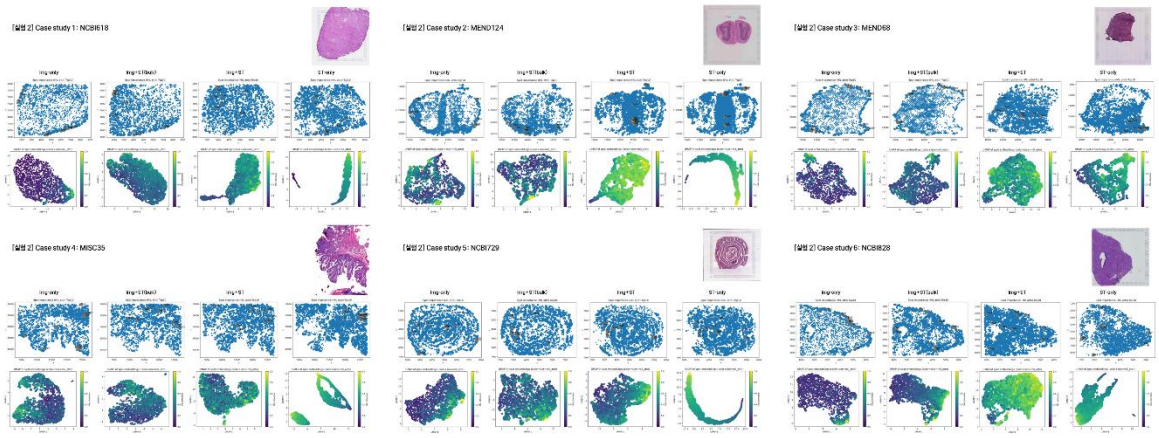


Figure 2. XAI 모듈 출력 예시. 샘플별 모델이 출력한 spot importance의 scatter plot과(위) 모델 내부 representation 구조(아래)이 UMAP.

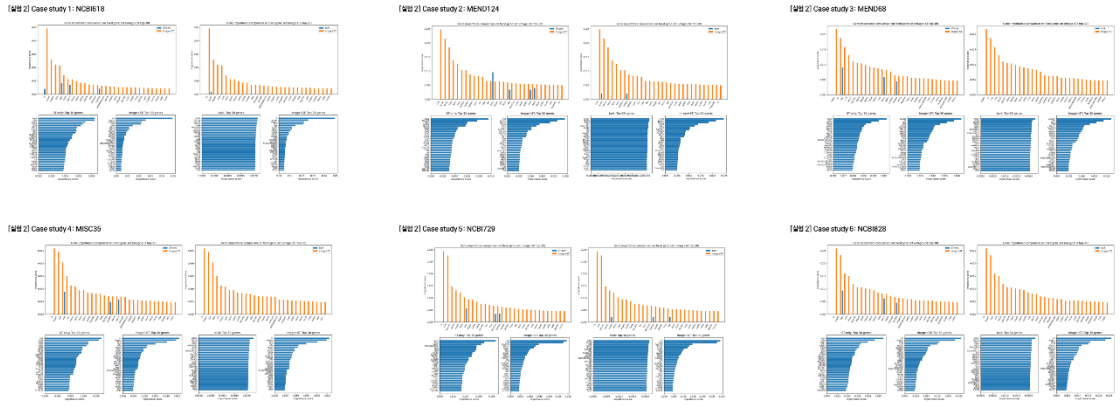


Figure 3. XAI 모듈 출력 예시. 샘플별 모델이 출력한 gene importance 그래프.

이어 위 프레임워크를 기반으로 두 가지 관점의 분석을 수행한다. 첫째, ST를 멀티모달 입력으로 사용할 때 효과적인 활용 방식을 규명하기 위해 인코더 구조에 따른 성능 비교 실험을 수행하고, 각 모듈 단계에서의 임베딩 변화를 분석하여 구조별 표현 학습의 차이를 해석한다. 둘째, 멀티모달 설정에서 ST의 실질적인 기여도를 분석하기 위해 모달리티 조합에 따른 성능 및 임베딩 비교 실험을 수행한다.

2.5.2. 인코더 구조에 따른 종양 예측 성능 비교 실험 (실험 1)

Fig. 1의 전체 모델 구조를 기반으로, ST 인코더를 late fusion 방식으로 구성한 Ver1과 early fusion 방식으로 구성한 Ver2를 비교한다. 각 버전의 모듈 단위 구조는 Figure 4에 제시되어 있다.

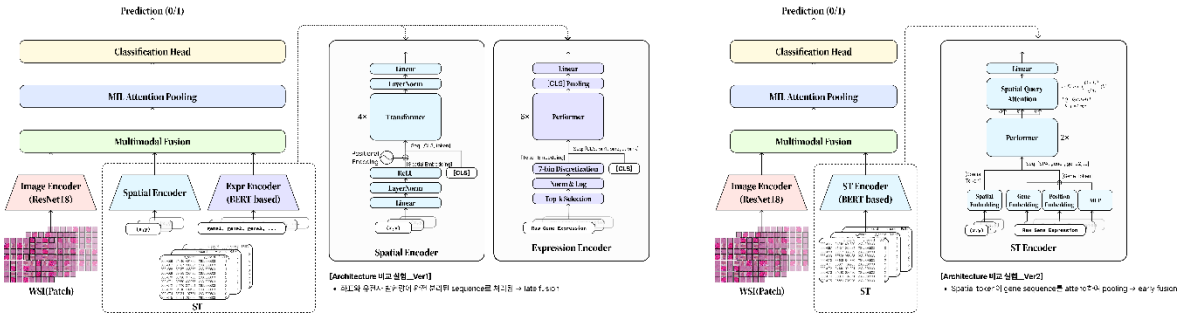


Figure 4. 인코더 구조 비교 실험의 모델 구조도. (좌측)좌표와 유전자 데이터를 완전 분리하는 late fusion 디자인(Ver1), (우측)좌표를 query로 유전자 데이터에 attention을 취하는 early fusion 디자인(Ver2)

Ver 1은 좌표 정보를 입력받는 ST 인코더와 유전자 발현 데이터를 입력받는 expression 인코더를 분리하여 구성하고, 각 인코더에서 생성된 representation을 결합하는 late fusion 방식으로 구현한다. 반면, Ver 2는 좌표와 발현량이 결합된 원시 ST 데이터를 하나의 ST 인코더에 입력으로 사용하고, 좌표 정보를 query로 활용하는 attention 메커니즘을 통해 representation을 생성한 뒤 이를 이미지 representation과 결합하는 early fusion 방식으로 구현한다. 이때 ST 인코더는 BERT 기반으로 구현하여, 다음 token을 예측하는 autoregressive 모델인 GPT 대비 데이터의 맥락 파악에 강점을 가지도록 한다.

이후 Ver 1과 Ver 2의 예측 성능(Val Acc, AUC, Precision/Recall/F1)을 비교하고, 각 모듈 단계에서 생성된 임베딩을 분석한다. 이를 통해 ST의 좌표 정보를 독립적인 입력으로 처리하는 late fusion 방식과, 좌표 정보를 기반으로 발현 데이터를 통합적으로 반영하는 early fusion 방식 중 어떤 접근이 ST 정보를 보다 효과적으로 활용하는지를 평가한다. 실험 1의 모듈별 구현과 실험 설계에 대한 자세한 내용은 이후 3.4.1에 서술한다.

2.5.3. 실험 1 결과

	Ver1	Ver2
attn	67.95	**76.92**
concat	70.51	**84.62**
sim	69.23	**80.77**
gate	74.36	**80.77**

Table 1. 네 가지 fusion 전략(attention-based, concatenation-based, similarity-based, gated fusion)에 대해 Ver1, Ver2 모델의 validation Accuracy(%)를 비교한 정량적 결과를 나타낸다. 각 행에서 가장 높은 성능을 기록한 경우를 bold로 표시하였다.

Ver2 모델은 Ver1 모델 대비 우수한 예측 성능을 보였으며(+6.4 ~ +14.1%p), 이는 유전자 표현

과 공간 좌표를 완전히 분리된 모달리티로 처리하는 설계가 본 데이터셋 및 실험 설정에서는 효과적으로 작동하지 않았음을 시사한다. 이러한 결과는 모달리티 간 상호작용을 late fusion에만 의존할 경우, 제한된 데이터 환경에서 표현 학습이 충분히 이루어지기 어렵다는 점을 보여준다.

정량적 성능 비교 결과를 종합하면, 모델 성능은 단순한 모달리티 수의 증가보다는 공간 정보를 어떠한 방식으로 통합하는지에 더 크게 의존함을 확인할 수 있다. Ver1 모델은 유전자 표현과 공간 정보를 독립적인 모달리티로 분리하여 처리함으로써 표현의 명확한 분리를 목표로 하였으나, 이러한 설계는 본 실험 환경에서는 오히려 성능 저하로 이어졌다.

반면 Ver2 모델은 공간 정보를 유전자 표현을 해석하는 조건 정보로 통합함으로써, 공간적 맥락을 반영한 표현 학습을 가능하게 하였다. 이는 공간 정보가 독립적인 예측 신호라기보다는 유전자 발현 패턴을 해석하는 데 필요한 맥락 정보로 작용할 때 더 효과적일 수 있음을 시사한다.

~~본 실험 결과는 방학 중 실시한 1차 실험의 결과로, validation accuracy에 대한 비교 결과만을 포함한다. 그 외 metric의 분석과 임베딩 추출 및 plot은 학기 중 진행할 예정이며, 모든 metric에서 ver2가 ver1을 outperform하고 임베딩이 잘 clustering되는 양상을 보일 것으로 예상된다.~~
Validation accuracy만으로 early fusion이 outperform한다는 주장을 뒷받침하기에 충분할 것으로 생각되어 추가 실험은 생략하였다.

2.5.4. 모달리티별 종양 예측 성능과 임베딩 비교 실험 (실험 2)

앞선 실험에서 ST 인코더 구조에 따른 성능 차이를 비교하였다면, 본 실험에서는 멀티모달 환경에서 ST를 활용하는 것이 실제로 유의미한 성능 향상으로 이어지는지를 분석한다. 특히, 단순히 멀티모달이 효과적인지뿐만 아니라, 다양한 생물학적 모달리티 중 ST가 가지는 상대적 유용성을 함께 평가하고자 한다.

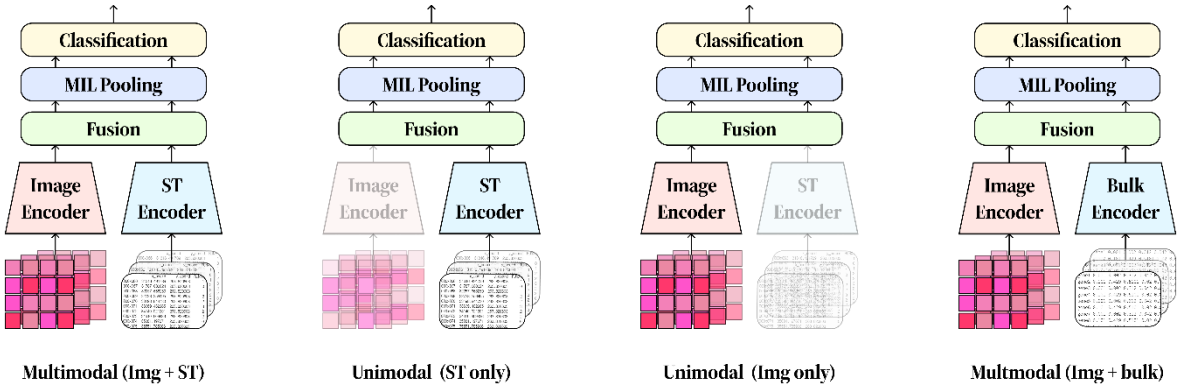


Figure 5. Modality ablation settings: 좌측부터 Img-ST 모델, ST-only 모델, Img-only 모델, Img+bulk 모델.

이를 위해 모델 구현 시 config/argument를 통해 각 인코더의 활성화 여부를 제어할 수 있도록 설계하고, ST only, Image only, ST+Image, Bulk+Image의 네 가지 모달리티 설정을 구성한다. 실험 2는 실험 1에서 가장 좋은 성능을 가졌던 "Ver2 인코더 + attention 기반 fusion" 모델 기반으로 진행되었으며, 각 설정에 대해 종양 예측 성능을 비교함과 동시에 학습 및 추론 과정에서 생성된 임베딩을 분석하였다. 이때 bulk 설정은 ST 데이터에서 좌표 정보를 제거하고 유전자 발현 데이터를 샘플 단위(WSI 단위)로 aggregate하여 가공한 데이터를 사용한다.

구체적으로, ST only, Image only, ST+Image 간 비교를 통해 "유니모달 대비 멀티모달 설정이 성능 향상에 기여하는지"를 검증하고, ST+Image와 Bulk+Image 간 비교를 통해 "멀티모달 구성 내에서 ST를 사용하는 것이 다른 분자적 표현(예: bulk) 대비 더 효과적인지"를 평가한다. 실험 2의 구현과 실험 설계에 대한 자세한 내용은 이후 3.4.2.에 서술한다.

2.5.5. 실험 2 결과

	val acc	val auc	precision	recall	f1-score
img-only	81.82%	0.8760	0.769	**0.909**	0.833
st-only	86.36%	0.8058	**1.000**	0.727	0.842
Img+st(bulk)	**90.91%**	0.9008	**1.000**	0.818	**0.900**
img+st	**90.91%**	**0.9835**	**1.000**	0.818	**0.900**

Table 2. 세 가지 모달리티 설정(image only, st only, 멀티모달(bulk), 멀티모달)으로 학습한 모델들의 예측 성능 metric. 각 행에서 가장 높은 성능을 기록한 경우를 bold로 표시하였다.

Table 2에서 확인할 수 있듯이, recall에서는 img-only 모델이 가장 높은 값을 보였으나, 나머지 핵심 성능 지표에서는 멀티모달 모델(img+st)이 단일 모달리티 모델을 상회하는 성능을 보였다. 특히 validation accuracy는 img-only 대비 +9.09%p, st-only 대비 +4.55%p 향상되었으며, ROC-AUC는 0.9835로 세 설정 중 가장 높은 판별 성능을 기록하였다.

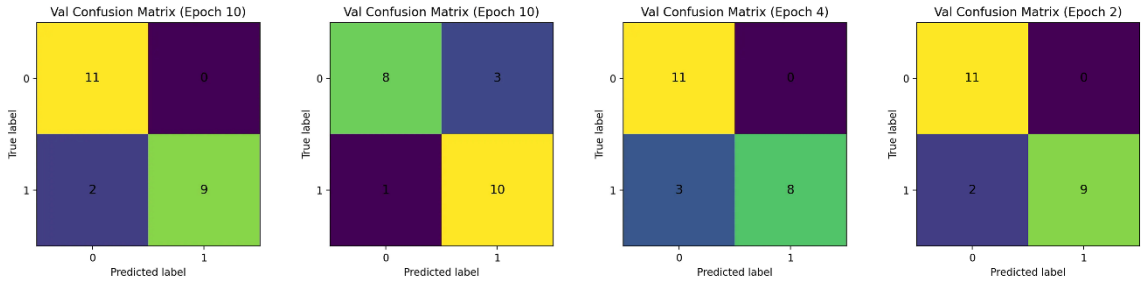


Figure 6. 좌측부터 멀티모달(full), Image-only, ST-only, 멀티모달(bulk) 설정에서의 confusion matrix.

Precision(정밀도)와 recall(재현율) 관점에서도 멀티모달 모델의 강점이 뚜렷하게 나타난다. img-only 모델은 높은 recall을 보였으나 precision이 상대적으로 낮아 false positive가 증가하는 경향을 보였으며, st-only 모델은 precision은 매우 높으나 recall이 낮아 일부 cancer 샘플을 놓치는 양상을 보였다. 반면, 멀티모달 모델(full multimodal, bulk multimodal)은 precision 1.000과 recall 0.818을 동시에 달성하여, 두 단일 모달리티의 장점을 효과적으로 결합한 균형 잡힌 예측 특성을 보였다. 그 결과 F1-score 역시 0.900으로 가장 높게 나타났다.

정량적 결과를 바탕으로, 각 모달리티가 예측 과정에서 보이는 특성을 분석하면 다음과 같은 경향을 확인할 수 있다. Figure 1에서 확인할 수 있듯, img-only 모델은 상대적으로 높은 recall을 기록하여 cancer 샘플을 민감하게 탐지하는 경향을 보였으나 precision이 낮아 false positive가 증가하는 양상을 보였다. 이는 이미지 기반 모델이 조직의 형태학적 특징을 효과적으로 포착하지만, 일부 정상 조직과 종양 조직 간의 미세한 차이를 구분하는 데 한계를 가질 수 있음을 시사한다.

반면 st-only 모델은 precision이 매우 높게 나타나 false positive를 효과적으로 억제하였으나, recall이 상대적으로 낮아 일부 cancer 샘플을 놓치는 경향을 보였다. 이는 공간 전사체 정보가 분자 수준의 특이성을 잘 반영하는 반면, 종양 내 이질성이나 국소적인 형태 변화를 충분히 포착하지 못할 가능성을 보여준다.

멀티모달 모델(img+st)은 이러한 두 단일 모달리티의 상반된 특성을 효과적으로 보완하였다. 이미지 정보가 제공하는 형태학적 단서와 공간 전사체가 제공하는 분자적 신호를 공동으로 활용함으로써, 높은 precision을 유지하면서도 recall 저하를 완화하는 균형 잡힌 예측 성능을 달성하였다. 이는 이미지와 공간 전사체가 서로 중복되지 않는 정보를 제공하며, 멀티모달 결합이 단일 모달리티 기반 예측에서 발생할 수 있는 편향을 완화하는 데 기여함을 의미한다.

~~실험 1과 마찬가지로 본 실험의 결과 또한 방학 기간 중 수행된 1차 실험의 결과로, bulk 설정에 대한 실험은 학기 중 진행할 예정이다. 실험 1에서 좌표를 query로 하여 발현량을 참조하는 것이 예측 성능 향상에 핵심적이었던 점, 실험 2에서 유니모달 설정보다 멀티모달 설정에서의 모델 performance가 우수했던 점을 고려하면 ST 기반 멀티모달 모델 학습에 좌표 정보가 중요한 정보를 한다는 것을 추측할 수 있고, 따라서 좌표 column이 제거된 bulk + image 모델은 원시 ST 기반의 full 모델 대비 성능이 하락할 것으로 예측 가능하다.~~

멀티모달 bulk 모델의 경우 단일 모달리티 모델을 outperform하였지만(val acc=90.91%), 멀티모달 full 모델 대비 AUC가 감소하였다. 이는 좌표 정보를 제거함에 따라 양성/음성 클래스의 분류 성능이 하락하였음을 의미한다. 따라서 ST의 좌표가 모델 성능에 핵심적인 역할을 한다는 1차 보고서의 추측이 타당하며, 이는 팀에서 주장하는 다음 두 가지 가설인 (1) ST 데이터 기반의 멀티모달 학습이 단일 모달리티 학습보다 적합하고, (2) 이때, 두 모달리티를 연결하는 bridge로서 ST의 좌표 데이터가 critical한 역할을 한다.를 뒷받침한다.

2.5.6. Explainability 모듈 및 case study

모델의 Explainability 보장을 위해 XAI 모듈을 도입 및 구현하였다. 이는 Multimodal Attention 기반으로 구현되였으며, 이를 통하여 Gene Importance, Spot Importance를 가시적으로 확인할 수 있게 되었다. 아래에서는 그 특성을 정성적으로 분석하고, WSI와 ST를 결합한 아키텍처가 설명 가능성 측면에서 가지는 독자적인 장점을 조사했다.

Baseline model(concat, early fusion)으로 3개 장기(뇌, 장, 간)에서 각각 정상(Healthy) 및 암(Cancer) 샘플을 하나씩 선정하여 총 6개의 샘플을 분석하였으며, 설명력의 차이가 입력 모달리티와 퓨전 구조에서 기인함을 보장하기 위해 모델 가중치를 고정한 상태에서 입력 모달리티 구성만 변경하며 진행되었다. XAI 모듈의 출력은 (1) 예측 결과 요약(logits & probs), (2) spot embedding을 투영한 UMAP, (3) MIL attention 기반 Spot Importance, (4) ST Encoder 기반 Gene Importance의 네 가지이다.

Case Study: NCBI618

멀티모달 설명 가능성의 장점이 가장 명확하게 드러난 간암 샘플(NCBI618)을 중심으로 세부 분석을 수행하였다. 해당 샘플은 Image-only, ST-only 모델에서 정답을 맞히지 못하였으나 baseline 모델에서는 정답을 맞힌 대표적 사례이다. 해당 샘플에 대해 XAI 모듈의 출력을 정성적으로 분석함으로써 결과를 설명한다.

1) Spot Embedding UMAP

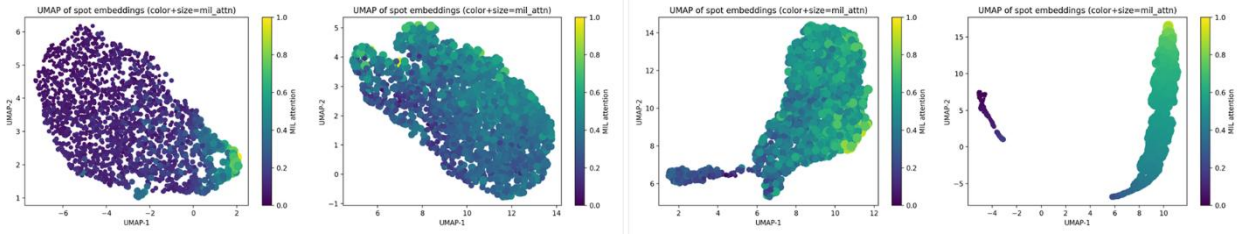


Figure 7. 모달리티별 spot 임베딩의 UMAP 시각화. 좌측부터 Image-only, 멀티모달(bulk), 멀티모달(full), ST-only 모델의 UMAP이다.

Attention이 Representation Space에서 어떻게 조직화되는지 확인하기 위해 UMAP 시각화를 수행하였다. Figure 7에서 확인할 수 있듯, ST-only에서는 High-Attention 스팟들이 좁은 궤적을 따라 배열되는 제한된 manifold 구조를 보이며, 이는 Attention 분포가 특정 표현축에 국한됨을 시사한다. Image-Only에서는 임베딩이 널리 분산되어 있고 High-Attention 스팟이 국소 영역에 집중되기는 하나, 전체 공간에서 일관된 구조적 조직화가 부족하다. 반면 멀티모달 모델들의 경우 Embedding 구조와 Attention 분포가 긴밀하게 정렬되며, 예측 관련 스팟들이 특정 영역에 군집화되는 양상을 띤다. 이는 Multimodal Fusion이 Representation Space 내에서 Attention의 구조화와 해석 가능성을 촉진함을 보여준다.

2) Spot Importance

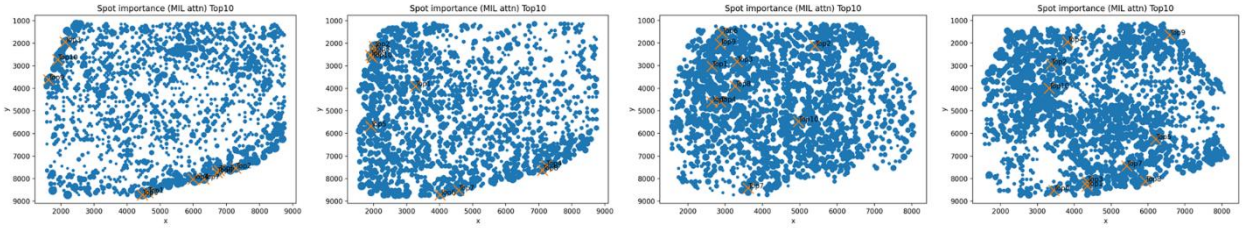


Figure 8. 모달리티별 spot importance의 시각화 scatter plot. 좌측부터 Image-only, 멀티모달(bulk), 멀티모달(full), ST-only 모델의 UMAP이다.

조직 공간 내 Attention 분포를 시각화하여 공간적 근거를 분석하였다. ST-only 모델에서는 high-attention spot이 분산되거나 협소한 경계에 갇혀 있어, 전사체 정보만으로는 충분한 공간적 맥락을 제공하기 어려움을 보여준다. 또한 Image-Only 모델에서는 모델 임베딩이 이미지 인코더로 사용한 ResNet의 inductive bias의 영향을 강하게 받음으로써 high-attention spot이 조직 경계 영역에 집중되며, 조직 전체에서 일관된 패턴을 형성하지 못한다.

반면 WSI+ST Multimodal 모델의 경우 High-Attention 스팟들이 특정 조직 하위 영역에 밀집되는 양상을 보이며, 이는 단일 모달리티에서는 관찰되지 않은 결과이다. 이러한 공간적 응집성은 형태학, 분자적 특성, 공간 정보를 통합 고려함으로써 더 명확한 Spatial Rationale을 제시할 수 있음을 입증한다. 특히, 형태학만으로는 식별이 어려운 영역들이 유전자 중요도 분석과 결합될 때 면역, 혈관, 기질 변화 등 종양 미세환경(TME)적 특징과 연관됨을 확인하였다.

3) Gene Importance

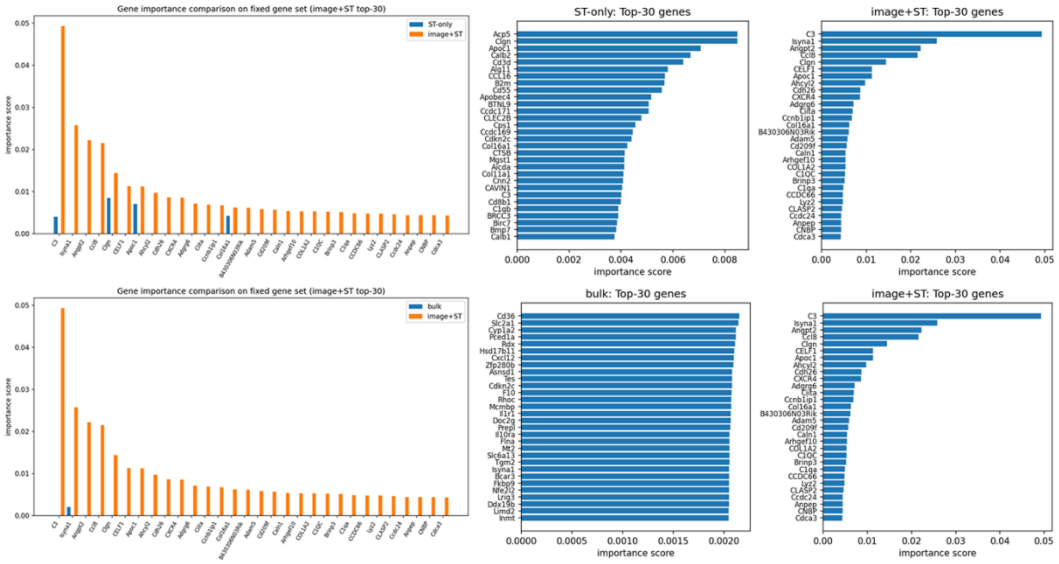


Figure 9. ST-only 모델, 멀티모달(full), 멀티모달(bulk) 각각의 gene importance 비교 그래프.

Attention 점수가 높은 스팟에 관련된 유전자를 분석한 결과는 위와 같다. WSI+ST Multimodal 모델의 경우 면역 반응 및 종양 미세환경 프로세스와 관련된 유전자(예: 보체 활성화 및 대식세포 신호 전달 관련 유전자(C3, C1qa, C1QC, Lyz2 등)와, 혈관 형성 및 기질 리모델링 관련 유전자(Angpt2, COL1A2, Col16a1 등)들이 높은 중요도를 기록하였다. 이는 모델이 단순히 암세포 밀집 지역뿐만 아니라 광범위한 미세환경적 문맥을 근거로 판단하고 있음을 시사한다. 반면 ST-only model에서는 주로 발현량 차이가 뚜렷한 유전자(Apoc1, Clgn, Acp5등)에 의존하며, 공간적·형태학적 문맥보다는 국소적인 발현 패턴에 의해 예측이 주도되는 한계를 보였다.

이를 통해 멀티모달 모델이 단일 모달리티 모델보다 더 정교하고 생물학적으로 타당한 예측 근거를 제시함을 확인하였으며, 특히 WSI+ST Multimodal은 종양 미세환경과 같은 복합적인 biomarker를 포착하는 데 핵심적인 역할을 수행한다는 것을 알 수 있었다.

Overall Clustering Quality

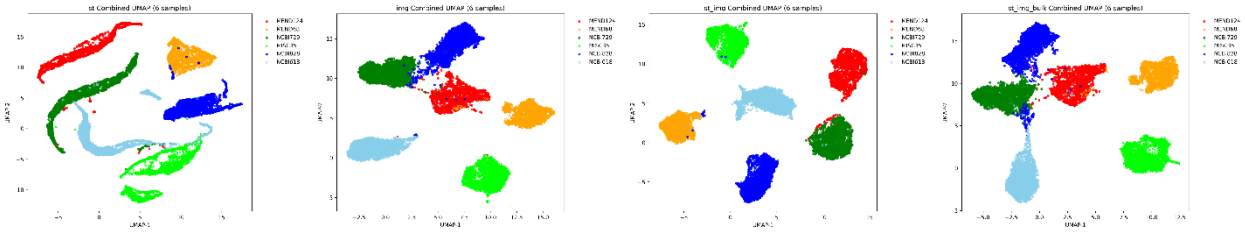


Figure 10. 모달리티별로 6개 test 샘플 임베딩을 clustering한 것. 좌측부터 ST-only, WSI-only, 멀티모달(full), 멀티모달(bulk)이다.

Case study를 진행한 NCBI618 샘플 포함, test set으로 사용한 6개 샘플에 대해 각 샘플의 임베딩 UMAP을 공통 space에 투영하여 clustering 품질을 평가하였다. Figure 10에서 확인할 수 있듯, ST-only 모델은 각각의 UMAP이 선형 manifold로 붕괴하였을 뿐 아니라 샘플별로 명확히 clustering되지 않았다. Img-only 모델은 UMAP이 붕괴하지는 않았으나, MEND124(red), NCBI729(green), NCBI828(blue)의 세 개 샘플 간 overlap되는 영역이 존재함으로써 해당 샘플들의 결정 경계가 불분명하다.

멀티모달(bulk) 모델 또한 Img-only 모델과 동일하게 MEND124(red), NCBI729(green), NCBI828(blue)의 세 개 샘플 간 overlapping이 관찰되었으며, 이는 ST 데이터에서 좌표 column을 drop한 결과 유전자 발현 데이터와 이미지 데이터를 연결하던 bridge가 사라졌고, 따라서 모델이 이미지 모달리티에 의존함으로써 Img-only와 유사한 임베딩을 생성하였음을 시사한다.

반면 full 멀티모달 모델은 (1) 샘플별 임베딩이 선형 manifold로 붕괴되지도, (2) 샘플 간 overlapping이 발생하지도 않아 전체 모달리티 option 중 clustering quality가 가장 우수하였으며, 이를 통해 좌표 정보를 보존한 멀티모달 fusion이 실제 case에서 모델의 성능을 유의미하게 leverage하고 있음을 확인할 수 있다.

3. Project Design

3.1. 요구사항 정의

3.1.1. 기능 요구사항

가장 먼저, 모델은 WSI와 ST 데이터를 입력으로 받아 처리할 수 있어야 한다. 각 데이터는 서로 다른 모달리티이므로 이를 고려하여 input으로 받을 수 있도록 설계한다.

이후 두 모달리티의 특성을 반영해 feature extraction을 할 수 있도록하는 모듈을 포함해야 하며, 모달리티 간의 정보를 효과적으로 통합하기 위한 멀티모달 학습 구조를 가져야 한다. 이 때, ablation 방식으로 모달리티 실험과 fusion 실험을 진행해야 하므로, 모달리티를 on/off 하거나 fusion layer를 변경할 수 있도록 설계되어야 한다.

최종적으로는 환자의 tumor 여부를 prediction 하여 Healthy(0) 또는 Cancer(1)의 이진 분류 예측을 수행해야 하며, 예측에 대한 모델의 의사결정 근거 또한 제공 가능해야 한다. 즉, ST에서는 gene importance를, WSI에서는 spot importance를 뽑아내며 이 두 정보를 좌표 기반으로 통합하는 XAI 모듈을 포함하여야 한다.

3.1.2. 비기능 요구사항

데이터 생산 비용과 관리에 대한 민감도로 인해 대규모 데이터셋을 구성하기 어렵고, 데이터셋 간 batch effect가 큰 바이오 데이터의 특성을 고려할 때, 모델은 소규모 데이터 환경에서도 안정적으로 학습 가능해야 하며 데이터셋에 관계없이 85% 이상의 정확도를 보이는 일반화 성능을 가져야 한다.

또, 실제 임상에서 사용되는 ST, WSI 데이터는 샘플 하나가 매우 용량이 크므로 계산 효율성을 고려해 합리적인 학습 시간 및 메모리 사용량을 유지해야 한다.

3.1.3. 요구사항 명세

- 입력 데이터: WSI(.tiff/.h5 내부에 포함), ST(.h5ad) 데이터

- 출력 데이터: 모델 예측값(healthy/cancer), gene importance score, top-k gene, spot importance score, top-k spot, (실험용)embedding, UMAP, etc.

3.2. 전체 시스템 구성

3.2.1. 데이터셋 구성

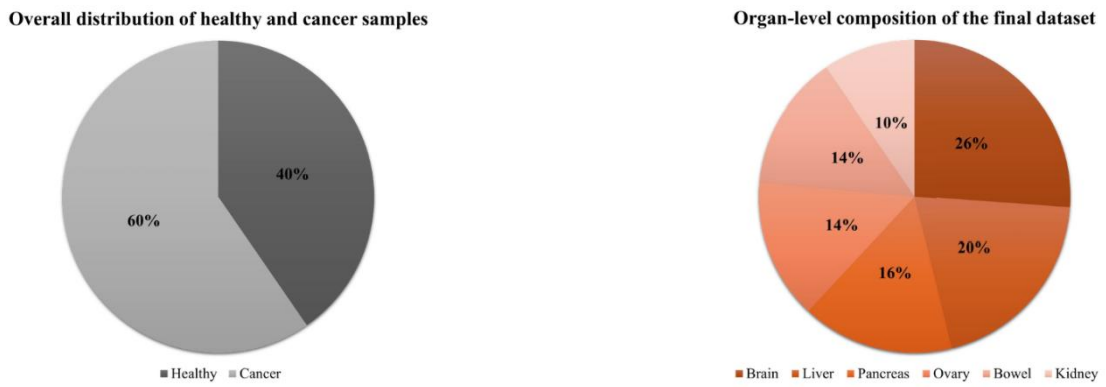


Figure 11. 연구에 사용한 데이터셋의 label별 클래스 분포.

Histology-based Expression STatistics (HEST) 벤치마크와 Spatial Transcriptomics Image (STimage) 데이터베이스로부터 샘플을 통합하여 포괄적인 멀티모달 공간 전사체 데이터셋을 구축하였다. 이러한 통합 전략은 다양한 조직 유형과 병리학적 상태 전반에 걸쳐 데이터의 품질과 일관성을 유지하면서 데이터셋의 다양성을 극대화하는 것을 목표로 한다.

- HEST-1K 데이터셋

HEST 벤치마크는 다양한 조직 유형과 병리학적 상태에 대한 조직학 이미지가 결합된 공간 전사체 데이터를 표준화된 형태로 제공한다. 생물학적 다양성을 충분히 확보하기 위해 세 가지 주요 기관계(뇌, 간, 장 조직)의 샘플을 선택하였으며, 총 156개의 샘플을 유지하였다.

- STimage-1K4M

STimage 데이터셋으로 난소, 췌장, 신장 조직의 샘플을 추가적으로 포함하였다. 그러나 STimage는 HEST와 달리 사전 추출된 패치 데이터를 제공하지 않고 전체 슬라이드 이미지(WSI)만 제공한다는 점이 주요한 도전 과제였다. 이를 해결하고 HEST 데이터 형식과의 일관성을 유지하기 위해, STimage 샘플에 대해 맞춤형 패치 추출 파이프라인을 적용하였다. 좌표 기반 패치 추출 방법을 구현하여, 모든 114개의 STimage 샘플에 대해 패치 데이터를 성공적으로 생성하였으며, 샘플당 평균 2,634 ± 1,389개의 유효 패치를 확보하였다.

- 데이터셋 통계

데이터 통합 이후, 본 연구의 데이터셋은 총 270개의 샘플로 구성되었다. 이후 정상과 암을 구분하는 이진 분류 과제를 위해, 분류가 어려운 모호하거나 중간 단계의 질병 상태를 가진 10개의 샘플을 제외하였다. 최종 데이터셋은 정상 샘플 105개(40.4%)와 암 샘플 155개(59.6%)로 구성되

며, 뇌, 간, 장, 난소, 췌장, 신장의 총 6개 주요 기관계를 포함한다.

이와 같은 기관 수준의 다양성은 서로 다른 해부학적 위치에서 나타나는 조직 구조, 세포 구성, 공간적 조직화의 큰 변이를 효과적으로 처리할 수 있는 일반화 가능한 멀티모달 모델을 개발하는 데 필수적이다.

3.2.2. 학습 설정

모든 모델은 AdamW optimizer를 사용하여 학습되었으며, learning rate은 5×10^{-4}, weight decay는 1×10^{-4}, gradient clipping은 norm 1.0으로 설정하고 총 10 epoch 동안 학습하였다. 이는 본 데이터셋 규모에서 수렴에 충분한 것으로 확인되었다. 또한, PyTorch의 기본 automatic mixed precision 기능을 활용하여 학습을 수행함으로써 메모리 사용량을 약 40% 절감하고, Tensor Core를 지원하는 GPU 환경에서 학습 속도를 약 2배 향상시켰다.

Whole-slide 기반 공간 전사체 데이터의 가변 길이 특성을 고려하여, WSI 단위에서 batch size를 1로 설정하여 한 번에 하나의 슬라이드를 처리하였다. 대신 4개의 슬라이드에 대해 gradient accumulation을 적용하여 유효 batch size를 4로 유지함으로써 안정적인 최적화를 달성하였다. 각 슬라이드 내부에서는 GPU 메모리 제약(NVIDIA A100 40GB)을 고려하여 500개의 spot 단위로 mini-batch를 구성하여 처리하였으며, 중간 임베딩은 CPU에 저장하여 peak 메모리 사용량을 최소화하였다. 또한, 200개 이상의 spot을 포함하는 슬라이드의 경우, 학습 시에는 무작위로 200개의 spot을 샘플링하고, 검증 시에는 모든 spot을 사용하여 메모리 초과를 방지하였다.

이미지 인코더(ResNet-18 backbone)는 학습 전반에 걸쳐 evaluation 모드로 고정하였으며, 최종 선형 projection head만 학습 가능하도록 설정하였다. 이는 소규모 의료 영상 데이터셋에서의 과적합을 방지하면서 ImageNet 사전학습된 특징을 효과적으로 활용하기 위함이다. 반면, 유전자 발현 인코더와 공간 인코더는 공간 전사체 데이터의 특성에 적응할 수 있도록 전체를 학습 가능하게 유지하였다.

재현성을 확보하기 위해 PyTorch, NumPy, Python의 random 모듈 전반에 걸쳐 random seed를 42로 설정하였으며, 클래스 비율(정상 40.4%, 종양 59.6%)을 유지하기 위해 stratified 방식으로 70/30의 train-validation 분할을 수행하였다.

3.2.3. 실험 설정

실험 1에서는 Ver1, Ver2 모델 대상으로 하여, 인코더 구성 및 fusion 전략 차이가 예측 성능에 미치는 영향을 분석하였다. 실험은 3.2.1에서 소개한 전체 데이터셋 기반으로 진행되었고, 이를 stratified sampling을 통해 70%의 학습 데이터와 30%의 검증 데이터로 분할하였으며, 실험 전반

에서 동일한 학습 프로토콜과 random seed가 사용되었다. 모든 모델은 healthy와 cancer를 구분하는 이진 분류 과제로 학습되었다. 본 실험에서는 아키텍처 설계에 따른 전반적인 성능 경향을 비교하는 데 초점을 두어 validation accuracy를 주요 평가 지표로 사용하였다.

데이터셋의 disease status(Healthy/Cancer) 분포와 장기 imbalance 문제를 고려하여, 실험 2는 Brain, Bowel, Liver의 세 장기를 포함하는 총 73개 샘플을 대상으로 수행되었으며, 클래스 분포는 Healthy 35개(47.9%), Cancer 38개(52.1%)로 비교적 균형을 이루도록 구성되었다. 데이터셋은 stratified sampling을 통해 70%의 학습 데이터와 30%의 검증 데이터로 분할되었다.

3.3. 주요 엔진 및 기능 설계

Note: 3.3. 전체가 2차 보고서에서 새로 추가된 내용이나, 가독성을 위해 이후 텍스트에서는 색상 구분을 생략한다.

본 프로젝트의 주요 "엔진"은 WSI와 ST를 입력받아 학습, 실험, 해석을 수행하는 연구용 모듈 단위로 구성된다. 전체 파이프라인은 데이터 로딩 및 정렬 모듈, 멀티모달 인코딩 및 fusion 모듈, 학습 및 평가 모듈, 모달리티 ablation 모듈, XAI 분석 모듈로 나눌 수 있다. 각 모듈은 독립적으로 실험 조건을 바꾸어 실행할 수 있도록 설계되었으며, 모델 구조 비교, 모달리티 비교, 설명 가능성 분석을 동일한 데이터 흐름 위에서 수행하도록 구성하였다. 전체 모델 구조와 실험 목적은 2.5절의 주요 기능 리스트에, 세부 구현 방식은 3.4절의 주요 기능 구현에 자세히 서술하며, 주요 모듈 및 기능의 overview는 다음과 같다.

1) <u>데이터 정렬 및 loading 모듈:</u>

ST의 gene expression 및 spatial coordinate와 WSI patch를 barcode 기준으로 정렬하여, spot 단위의 멀티모달 입력을 구성한다. 데이터셋 구성과 학습 설정은 3.2절에서, dataloader 구현 방식은 3.4.2절에서 다룬다.

2) <u>멀티모달 인코딩 및 fusion 모듈:</u>

Image 인코더와 ST 인코더에서 추출한 spot-level representation을 다양한 fusion 방식으로 결합하여 WSI-level 종양 예측에 사용한다. 본 연구에서는 여러 fusion 방식을 비교하여 두 모달리티의 상호작용을 가장 효과적으로 반영하는 구조를 탐색하였다. 각 인코더 구조와 fusion 방식의 세부 구현은 3.4.1절에 서술한다.

3) <u>학습 및 평가 모듈</u>:

모델 설정, optimizer, mixed precision, gradient accumulation, checkpoint 저장, 성능 지표 계산을 담당한다. 학습 설정은 3.2.2절에, fusion 실험 학습 구현은 3.4.1절에 제시한다.

4) 모달리티 ablation 모듈:

use_image, use_st 플래그를 통해 image-only, ST-only, image+ST 조건을 동일한 파이프라인에서 비교할 수 있도록 한다. 해당 구현은 3.4.2절에서 자세히 설명한다.

5) XAI 모듈:

Inference 결과로부터 prediction probability, spot embedding UMAP, MIL attention 기반 spot importance, gene attention 기반 gene importance를 생성하며, 이를 통해 모델이 어떤 조직 영역과 유전자 정보를 근거로 종양 예측을 수행했는지 확인할 수 있다. XAI 출력의 구체적인 코드와 저장 방식은 3.4.3절에 정리한다.

정리하면, 본 프로젝트의 주요 엔진은 데이터 정렬, 멀티모달 표현 학습, fusion 실험, 모달리티 ablation, XAI 분석을 담당하는 연구용 모듈들로 구성된다. 2.5절에서는 각 엔진이 제공하는 기능과 실험 목적을 중심으로 설명하고, 3.4절에서는 해당 기능들이 실제 코드 수준에서 어떻게 구현되었는지를 fusion 실험, modality 실험, XAI 모듈로 나누어 상세히 기술한다.

3.4. 주요 기능의 구현

Note: 3.4. 전체가 2차 보고서에서 새로 추가된 내용이나, 가독성을 위해 이후 텍스트에서는 색상 구분을 생략한다.

3.4.1. Fusion 모듈 설계 및 실험 구현

멀티모달 학습에서는 "어떻게 두 modality를 합치느냐"가 성능에 큰 영향을 준다. 단순히 feature를 이어붙이는 것(concat)과, 두 modality 간 관계를 명시적으로 모델링하는 것(attention, gated) 사이에는 분명한 차이가 존재한다. MOSAIC-ST는 WSI와 ST 데이터를 통합하는 프레임워크이고, 따라서 spot 단위에서 이미지 특징과 ST 특징을 추출한 뒤 이를 하나의 표현으로 합치는 fusion 단계가 핵심이며, 이 fusion 전략을 어떻게 설계하느냐에 따라 같은 인코더 구조에서도 성능이 크게 달라진다는 점을 본 실험의 motivation으로 한다.

실험 1에서는 fusion 전략 실험을 인코더 구조와 함께 비교하는데, 이는 두 요소가 독립적이지 않기 때문이다. ST 정보를 어떻게 인코딩하느냐에 따라 fusion 단계에서 요구되는 것이 달라지므로 본 실험에서 이를 검증한다.

1) 인코더 구조 비교

- Ver1: Triple-Encoder 구조

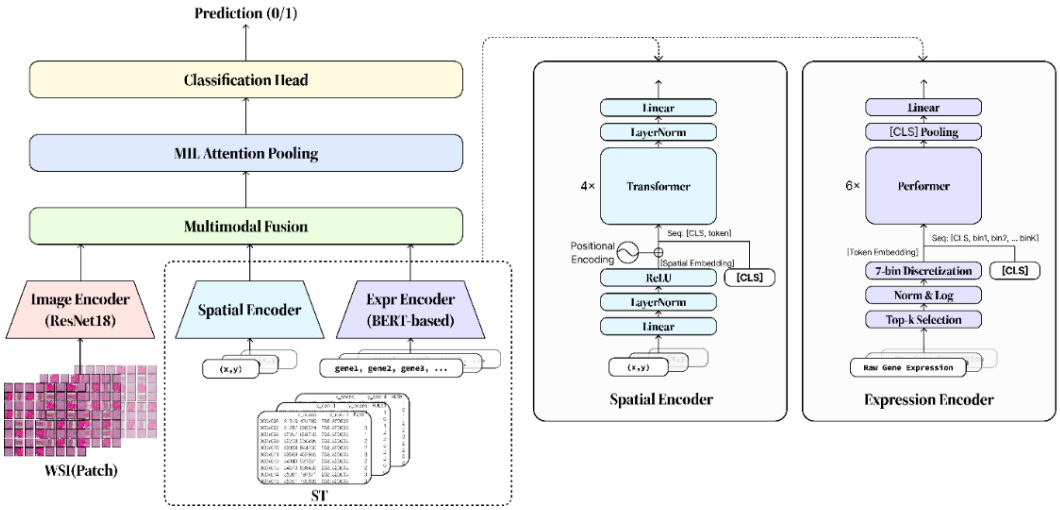


Figure 12. 인코더 구조 비교 실험(실험 1)의 Ver1 모델 구조도. 좌표와 유전자 발현 데이터가 완전히 분리된 상태로 representation을 각각 생성한 뒤, 이후 fuse하는 late fuson 형태이다.

Figure 8과 같이, ver1 구조에서는 유전자 발현 정보와 공간 좌표 정보를 완전히 분리된 두 개의 인코더로 처리한다. 결과적으로 이미지, 유전자, 좌표 세 가지 표현이 fusion 단계에 전달된다.

```
class TripleEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.img_encoder        = ImageEncoder()        # ResNet-18
        self.gene_encoder       = GeneEncoder()         # scBERT-inspired Performer
        self.spatial_encoder    = SpatialEncoder()      # Transformer (2D coord → 256-dim)

    def forward(self, image, gene_expr, spatial_coord):
        v    = self.img_encoder(image)                  # (N, 256)
        h_gene = self.gene_encoder(gene_expr)           # (N, 256)
        h_sp   = self.spatial_encoder(spatial_coord)    # (N, 256)
        return v, h_gene, h_sp
```

유전자 인코더는 scBERT 방식에 착안해서 유전자를 토큰으로 다루는 Performer 구조를 사용한다. 발현값을 7개 구간으로 이산화하여 lookup table로 임베딩한다.

```
class GeneEncoder(nn.Module):
    def __init__(self, n_genes=2000, top_k=512, d_model=256):
        super().__init__()
        self.top_k = top_k
        self.bin_emb = nn.Embedding(7, d_model)     # 발현값 이산화: 0~6 구간
        self.id_emb  = nn.Embedding(n_genes, d_model)
        self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))
```

```
        self.performer = PerformerEncoder(depth=6, heads=8, dim=d_model)
        self.proj = nn.Linear(d_model, d_model)

    def forward(self, gene_expr):
        topk_vals, topk_idx = gene_expr.topk(self.top_k, dim=-1)

        # 발현값 이산화
        bins = torch.bucketize(
            torch.log1p(topk_vals),
            boundaries=torch.linspace(0, 5, 6)
        ).clamp(0, 6)                                   # (N, K)

        tokens = self.id_emb(topk_idx) + self.bin_emb(bins)     # (N, K, D)
        cls = self.cls_token.expand(tokens.size(0), -1, -1)     # (N, 1, D)
        seq = torch.cat([cls, tokens], dim=1)                   # (N, K+1, D)

        out = self.performer(seq)
        return self.proj(out[:, 0])                     # CLS pooling → (N, D)
```

- <u>Ver2: Spatial-aware Encoder 구조</u>

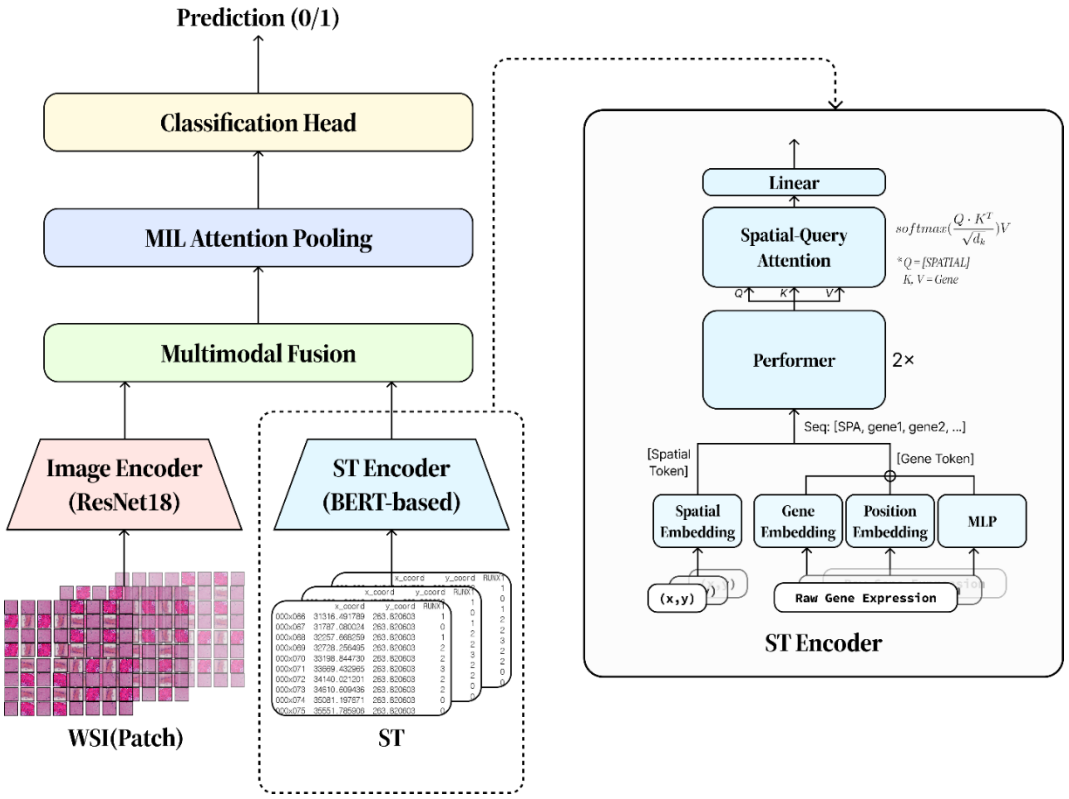


Figure 13. 인코더 구조 비교 실험(실험 1)의 Ver2 모델 구조도. 좌표 데이터와 유전자 발현 데이터가 통합된 원시 ST 데이터를 단일 인코더에 입력받으며, 좌표를 query로 하는 spatial-query attention으로 representation을 생성하는 early fusion 형태이다.

Figure 9와 같이, ver2 구조에서는 공간 좌표를 독립된 인코더로 분리하지 않고, 유전자 인코딩 과정 내부에 공간 정보를 통합한다. 즉, spatial token과 gene token이 함께 Transformer를 통과하며 self-attention을 공유한다.

```
class SpatialAwareSTEncoder(nn.Module):
    def __init__(self, n_genes=2000, top_k=512, d_model=256):
        super().__init__()
        self.top_k = top_k
        self.gene_id_emb   = nn.Embedding(n_genes, d_model)
        self.gene_pos_emb = nn.Embedding(top_k, d_model)
        self.expr_proj      = nn.Sequential(
            nn.Linear(1, d_model), nn.LayerNorm(d_model), nn.GELU()
        )
        self.spatial_proj = nn.Linear(2, d_model)     # (x, y) → 256-dim

        encoder_layer = nn.TransformerEncoderLayer(
```

```
            d_model=d_model, nhead=8, batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=4)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, gene_expr, spatial_coord):
        topk_vals, topk_idx = gene_expr.topk(self.top_k, dim=-1)

        # gene token = identity + position + expression value
        gene_tokens = (
            self.gene_id_emb(topk_idx)                                  # (N, K, D)
            + self.gene_pos_emb(torch.arange(self.top_k).to(topk_idx.device))
            + self.expr_proj(topk_vals.unsqueeze(-1))
        )

        # spatial token
        s_token = self.spatial_proj(spatial_coord).unsqueeze(1)     # (N, 1, D)

        # [spatial | gene] 통합 후 Transformer
        seq = torch.cat([s_token, gene_tokens], dim=1)              # (N, K+1, D)
        out = self.transformer(seq)                                  # (N, K+1, D)

        # spatial token을 query로, gene token들과 cross-attention
        q = out[:, 0:1, :]      # (N, 1, D)
        k = v = out[:, 1:, :] # (N, K, D)
        d = q.size(-1)
        attn = torch.softmax(q @ k.transpose(-2, -1) / d**0.5, dim=-1)
        h_st = (attn @ v).squeeze(1)                                 # (N, D)

        return self.out_proj(h_st)
```

결국 Ver1과 Ver2 구조의 핵심적인 차이는 공간 정보를 독립 신호로 다루는지, 유전자 표현을 해석하는 조건 정보로 다루는지이다.

2) Fusion 전략 비교

하기 4가지 fusion 옵션은 각 spot에서 이미지 representation v와 ST representation S를 전달받아 통합된 represesntation z를 만드는 방식을 설명한다.

- <u>Concatenation-based MLP</u>

```
class ConcatFusion(nn.Module):
```

```
    def __init__(self, d=256):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(d * 2, d),
            nn.LayerNorm(d),
            nn.ReLU(),
            nn.Linear(d, d)
        )

    def forward(self, v, s):
        return self.mlp(torch.cat([v, s], dim=-1))
```

Concatenation 방식은 4가지 option 중 가장 단순한 방법으로, 두 representation을 이어붙인 뒤 MLP로 압축한다. 따라서 두 modality 간 명시적 관계 모델링은 수행하지 않는다.

- Attention-based Fusion

```
class AttentionFusion(nn.Module):
    def __init__(self, d=256, n_heads=4):
        super().__init__()
        self.cross_attn = nn.MultiheadAttention(d, n_heads, batch_first=True)
        self.norm = nn.LayerNorm(d)
        self.proj = nn.Linear(d, d)

    def forward(self, v, s):
        # v를 query, s를 key/value로 — 이미지가 ST에서 무엇을 참조할지 학습
        v_ = v.unsqueeze(1)     # (N, 1, D)
        s_ = s.unsqueeze(1)     # (N, 1, D)
        out, _ = self.cross_attn(v_, s_, s_)
        out = self.norm(out.squeeze(1) + v)     # residual
        return self.proj(out)
```

Attention 기반 option은 이미지 representation이 ST representation을 참조하는 cross-attention으로 구현한다. 따라서 concatenation 기반과 달리 두 모달리티 간의 상호작용을 명시적으로 모델링한다.

- Similarity-based Fusion

```
class SimilarityFusion(nn.Module):
    def __init__(self, d=256):
        super().__init__()
        self.proj = nn.Sequential(
            nn.Linear(d * 3, d),
```

```
            nn.LayerNorm(d),
            nn.ReLU(),
            nn.Linear(d, d)
        )

    def forward(self, v, s):
        elem_prod = v * s                                          # element-wise product
        abs_diff  = (v - s).abs()                                # absolute difference
        cos_sim   = F.cosine_similarity(v, s, dim=-1, eps=1e-8)   # (N,)
        cos_sim   = cos_sim.unsqueeze(-1).expand_as(v)              # (N, D)

        combined = torch.cat([elem_prod, abs_diff, cos_sim], dim=-1)  # (N, 3D)
        return self.proj(combined)
```

Similarity based 방식은 두 representation 사이의 유사도를 다양한 방법으로 계산하여 조합하는 방식으로서, 두 모달리티가 얼마나 "일치"하는지를 명시적으로 포착하는 기법이다.

- <u>Gated Fusion</u>

```
class GatedFusion(nn.Module):
    def __init__(self, d=256):
        super().__init__()
        self.gate_v = nn.Sequential(nn.Linear(d * 2, d), nn.Sigmoid())
        self.gate_s = nn.Sequential(nn.Linear(d * 2, d), nn.Sigmoid())
        self.proj   = nn.Linear(d, d)

    def forward(self, v, s):
        combined = torch.cat([v, s], dim=-1)
        g_v = self.gate_v(combined)   # 이미지 특징의 기여도
        g_s = self.gate_s(combined)   # ST 특징의 기여도
        z   = g_v * v + g_s * s
        return self.proj(z)
```

Gated fusion 방식은 각 모달리티의 기여도를 입력에 따라 동적으로 조절한다. 즉, 샘플마다 이미지와 ST 중 어느 쪽이 더 신뢰할 만한지를 학습한다.

3.4.2. Modality 실험 구현

본 프로젝트에서는 이미지 모달리티와 공간 전사체(ST, Spatial Transcriptomics) 모달리티의 기여도를 비교하기 위해 encoder on/off 방식의 ablation 실험 구조를 구현하였다. Default 모델 코드 *model.py*는 항상 이미지 encoder와 ST encoder를 모두 사용한 뒤 fusion module을 거쳐 MIL

classification을 수행하는 구조였으나, ablation 버전에서는 *use_image*, *use_st* 플래그를 추가하여 이미지 단독, ST 단독, 이미지+ST 멀티모달 세 가지 조건을 동일한 학습 및 평가 파이프라인에서 실행할 수 있도록 하였다.

1) 모델 코드

기본 모델인 *models/models.py*의 *MultimodalMILModel*은 *ImageEncoder*, *SpatialEncoder*, *SpotFusionModule*, *MILAttentionPooling*, *classifier*를 모두 생성하고, forward에서 항상 이미지 feature와 ST feature를 계산한 뒤 fusion을 수행한다. 즉, 기본 구조는 다음과 같다.

image patch → ImageEncoder → img_head

gene expression + coords → SpatialSTEncoder

img_feat + st_feat → SpotFusionModule

spot embeddings → MILAttentionPooling → classifier

반면 ablation-support 모델인 *models/model_ablation.py*에서는 *MultiModalMILModel* 생성자에 *use_image=True, use_st=True* 플래그가 추가되었다. 이 두 값은 최소 하나 이상 켜져(True) 있어야 하며, 둘 다 꺼지는 잘못된 실험 설정은 *assert use_image or use_st*로 차단한다. 모델 내부에서는 이들 플래그에 따라 조건부로 인코더를 생성한다.

또한 fusion 모듈은 두 모달리티가 모두 활성화된 경우에만 생성된다. 이미지 단독(Img-only 모델) 또는 ST 단독(ST-only 모델) 실험에서는 fusion이 필요하지 않으므로, *self.fusion=None*으로 둔다. Forward 단계에서도 동일한 라우팅을 적용하며, 위 내용들을 정리하면 다음과 같다.

image + ST → fusion(img_feat, st_feat)

image only → spot_embeds = img_feat

ST only → spot_embeds = st_feat

즉, ablation 모델은 최종적으로 항상 동일한 형태의 *spot_embeds*를 MIL pooling에 전달하되, 그 *spot_embeds*가 어떤 인코더에서 왔는지만이 실험 조건에 따라 달라지도록 설계되었다. 덕분에 encoder 이후의 MIL pooling과 classifier는 모든 실험에서 동일하게 유지된다.

2) YAML 설정

Ablation 실험용 설정은 *configs/train_ablation.yaml*에 분리되어 있다. 기존 *configs/train.yaml*에는 멀티모달 학습 설정만 존재하지만, ablation을 위해 *model* 항목 아래에 다음 두 옵션이 추가되었다.

```
model:
```

```
  fusion_option: concat
  top_k_genes: 512

  use_image: true
  use_st: true
```

위 값을 바꿈으로써, 동일한 코드를 이용해 아래 세 가지 실험을 수행할 수 있다.

use_image: true, use_st: true → Image + ST 멀티모달

use_image: true, use_st: false → Image-only

use_image: false, use_st: true → ST-only

3) Train 코드

Ablation 학습은 *train_ablation.py*에서 수행된다. *load_config()* 함수는 *configs/train_ablation.yaml*을 읽고, *use_iamge*, *use_st* 값을 *CONFIG* dictionary에 저장한다.

```
"use_image": cfg["model"].get("use_image", True),
"use_st": cfg["model"].get("use_st", True),
```

이후 *assert CONFIG["use_image"] or CONFIG["use_st"]*로 최소 하나의 모달리티가 활성화되었는지 확인한다.

Train 코드의 핵심은 *encode_spots_chunkwise()* 함수이며, 이 함수는 하나의 WSI에 포함된 여러 spot을 batch 크기로 나누어 처리하면서, 설정된 모달리티에 필요한 입력만 사용한다.

use_image=True → batch["images"] 사용

use_st=True → batch["expr"], batch["coords"] 사용

즉, 각 chunk마다 아래와 같이 라우팅한다.

```
image + ST:
  img_encoder → img_head
  st_encoder
  fusion(img_feat, st_feat)

image only:
  img_encoder → img_head

ST only:
  st_encoder
```

이렇게 얻은 *spot_embeds*는 이후 모든 실험에서 동일하게 *model.mil_pooling()*과

*model.classifier()*로 전달된다. 즉, 모달리티 비교 실험에서 encoder와 fusion 입력만 달라지고, MIL pooling 및 classifier는 동일하게 유지되도록 구현하였다.

4) Test 코드

평가 및 XAI 분석용 ablation 코드는 *test_ablation.py*에 구현되어 있다.

```
parser.add_argument("--use_image", action="store_true")
parser.add_argument("--use_st", action="store_true")
```

```
images = batch["images"].to(args.device) if args.use_image else None expr = batch["expr"].to(args.device)
if args.use_st else None coords = batch["coords"].to(args.device) if args.use_st else None
```

학습은 YAML 기반으로 모달리티를 설정하지만, 테스트 코드는 command-line argument 방식으로 encoder on/off를 지정하며, defaut는 멀티모달로 하여 아무 옵션도 지정되지 않을 경우 멀티모달 실험이 진행되도록 처리하였다.

```
images = batch["images"].to(args.device) if args.use_image else None
expr   = batch["expr"].to(args.device)   if args.use_st else None
coords = batch["coords"].to(args.device) if args.use_st else None
```

테스트 시에도 모델 생성자에 동일하게 *use_image*, *use_st*가 전달된다. 또한 batch에서 필요한 모달리티만 GPU로 올리도록 구현하였다.

더불어 ST 모달리티가 활성화되어 있을 때만 gene attention을 반환하도록 *return_gene_attn=bool(args.use_st)*를 사용한다. 따라서 image-only 실험에서는 gene-level XAI를 계산하지 않고, ST 또는 multimodal 실험에서만 gene attention 기반 top gene 분석을 수행한다. 반대로 top patch 저장은 image 모달리티가 켜진 경우에만 수행된다.

5) DataLoader와 모달리티 정렬

DataLoader인 *dataset/loader.py*는 기본적으로 image patch, gene expression, spatial coordinate를 모두 반환한다. ST spot과 image patch는 barcode 기준으로 정렬되며, 반환되는 주요 항목은 다음과 같다.

images → image encoder 입력

expr → ST encoder의 gene expression 입력

coords → ST encoder의 spatial token 입력

label → WSI-level classification label

즉, dataloader는 모든 모달리티를 준비하지만, 실제 train 및 test 단계에서 *use_image*, *use_st* 플래그에 따라 필요한 입력만 선택적으로 사용한다. 이 방식은 데이터 split과 sample 구성을 동일하게 유지하면서 인코더의 on/off만을 수행하므로, 모달리티별 성능 비교의 공정성을 높인다.

6) 정리

결과적으로, 본 ablation 구현은 *use_image*와 *use_st* 두 개의 플래그를 중심으로 구성된다. YAML에서는 학습 실험 조건을 설정하고, train code는 해당 설정을 읽어 모델 생성 및 spot encoding 라우팅에 반영한다. test code에서는 command-line argument로 동일한 조건을 재현하며, 활성화된 모달리티에 맞춰 patch-level XAI 또는 gene-level XAI를 선택적으로 저장한다. 모델 내부에서는 encoder 생성, freeze 처리, fusion 생성, forward 라우팅이 모두 플래그 기반으로 동작하므로, image-only, ST-only, image+ST 실험을 동일한 MIL backbone 위에서 비교할 수 있도록 구현하였다.

3.4.3. XAI 모듈 구현

1차 보고서 분량에 추가하여, 모델의 Explainability 보장을 위해 XAI 모듈을 도입 및 구현하였다. XAI 모듈은 multimodal attention 기반으로 구현되며, 모델 inference 결과에서 prediction score, spot-level embedding, MIL attention, gene attention을 추출하여 저장하는 방식으로 작동한다. 모델은 forward 결과를 dictionary 형태로 반환하며, *logits, mil_attn, spot_embeds, gene_attn, gene_indices*를 후처리 코드에서 사용한다. 이를 통하여 gene importance와 spot importance를 가시적으로 확인할 수 있다.

Baseline model(concat, Ver2)으로 3개 장기(뇌, 장, 간)에서 각각 정상(Healthy) 및 암(Cancer) 샘플을 하나씩 선정하여 총 6개의 샘플을 분석하였으며, 설명력의 차이가 입력 모달리티와 퓨전 구조에서 기인함을 보장하기 위해 모델 가중치를 고정한 상태에서 입력 모달리티 구성만 변경하며 진행되었다. XAI 모듈은 주요 출력으로

- Logits & Probabilities 출력
- spot embedding을 투영한 UMAP
- MIL attention 기반 Spot Importance
- ST Encoder 기반 Gene Importance

의 네 가지를 가지며, 이때 (2) UMAP은 임베딩 분석을 통해 WSI+ST 아키텍처의 타당성을 보이기 위해 추가된 요소로서 임상적 정보가 아닌 <u>모델이 생성한 임베딩이 얼마나 잘 clustering되어 있는지</u>를 보인다.

1) Logits & Probabilities 출력

```
outputs = model(
    images,
    expr,
    coords,
```

```
    return_gene_attn=bool(args.use_st),
    return_spot_embeds=True
)

logits = outputs.get("logits", None)
mil_attn = outputs.get("mil_attn", None)
spot_embeds = outputs.get("spot_embeds", None)

gene_attn = outputs.get("gene_attn", None)
gene_indices = outputs.get("gene_indices", None)

probs = F.softmax(logits, dim=-1).detach().cpu().numpy().tolist()
pred = int(np.argmax(probs))

summary = {
    "sample_id": sample_id,
    "gt_label": label,
    "pred_label": pred,
    "probs": probs,
    "num_spots_used": n_spots,
    "use_image": bool(args.use_image),
    "use_st": bool(args.use_st),
    "mil_attn_available": mil_attn is not None,
    "spot_embeds_available": spot_embeds is not None,
    "gene_xai_available": (gene_attn is not None and gene_indices is not None),
}

with open(os.path.join(out_dir, "pred.json"), "w") as f:
    json.dump(summary, f, indent=2)
```

본 코드는 모델의 raw classification output인 *logits*를 softmax로 변환하여 클래스별 예측 확률을 계산하고, 가장 높은 확률의 클래스를 prediction으로 저장한다. 각 샘플별로 *pred.json*을 생성하여 정답 label, 예측 label, class probability, 사용된 모달리티 정보, XAI 정보 사용 가능 여부를 함께 기록하도록 구현하였다.

2) UMAP 출력

```
def compute_2d_embedding(X: np.ndarray, method: str = "umap", seed: int = 0):
    if method == "umap":
        try:
            import umap
```

```
            reducer = umap.UMAP(n_components=2, random_state=seed)
            return reducer.fit_transform(X)
        except Exception as e:
            print(f"[warn] UMAP not available ({e}). Falling back to PCA.")
            method = "pca"

    if method == "pca":
        from sklearn.decomposition import PCA
        return PCA(n_components=2, random_state=seed).fit_transform(X)

    raise ValueError(f"Unknown method: {method}")
```

```
if spot_embeds is not None:
    X = spot_embeds.detach().cpu().numpy()
    Z = compute_2d_embedding(X, method=args.embed_2d, seed=0)

    a = mil_attn.detach().cpu().numpy()
    a_min, a_max = float(a.min()), float(a.max())
    denom = (a_max - a_min) if (a_max - a_min) > 1e-12 else 1.0
    a_n = (a - a_min) / denom

    plt.figure()
    plt.scatter(Z[:, 0], Z[:, 1], s=10 + 200 * a_n)
    plt.title(f"{args.embed_2d.upper()} of spot embeddings (size=mil_attn)")
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, f"spot_embeds_{args.embed_2d}.png"), dpi=200)
    plt.close()

    plt.figure()
    plt.scatter(
        Z[:, 0], Z[:, 1],
        c=a_n,
        s=10 + 200 * a_n,
        cmap="viridis"
    )
    plt.colorbar(label="MIL attention")
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, f"spot_embeds_{args.embed_2d}_color.png"), dpi=200)
    plt.close()
```

위 코드는 모델이 생성한 spot-level embedding을 2차원으로 축소하여 시각화하는 부분이다. 기

본적으로 UMAP을 사용하고, UMAP 사용이 불가능한 경우 PCA로 대체되도록 구현하였다. 각 spot의 MIL attention 값을 점의 크기 또는 색상으로 반영하여, embedding space에서 중요한 spot들이 어떻게 분포하는지 확인할 수 있게 하였다.

3) Spot Importance 출력

```
mil_attn = mil_attn.view(-1)

top10 = torch.topk(
    mil_attn,
    k=min(10, n_spots)
).indices.detach().cpu().tolist()
```

```
def make_top_percent_mask(attn: torch.Tensor, top_percent: float = 0.6, min_points: int = 10):
    attn = attn.view(-1)
    N = attn.numel()

    q = 1.0 - float(top_percent)
    q = min(max(q, 0.0), 1.0)

    thr = torch.quantile(attn, q)
    mask = attn >= thr

    if mask.sum().item() < min_points:
        k = min(min_points, N)
        idx = torch.topk(attn, k=k).indices
        mask = torch.zeros(N, dtype=torch.bool, device=attn.device)
        mask[idx] = True

    return mask
```

```
if coords_raw is not None:
    mask70 = make_top_percent_mask(mil_attn, top_percent=0.7, min_points=20)

    plot_attention_scatter(
        coords_raw=coords_raw,
        attn=mil_attn.detach().cpu(),
        top10_idx=top10,
        out_path=os.path.join(out_dir, "patch_attn_scatter_top10.png"),
        title="Spot importance (MIL attn) Top10",
        mask=mask70
    )
```

위 코드는 MIL attention score를 spot importance로 해석하여 중요한 spot을 추출한다. *mil_attn*은 MIL pooling에서 각 spot이 WSI-level prediction에 얼마나 기여했는지를 나타내며, attention score가 높은 top-k spot을 선택한다. 또한 상위 attention 구간만 mask로 남긴 뒤 원래 spatial coordinate 위에 scatter plot으로 표시하여, 중요한 spot이 조직 내 어느 위치에 분포하는지 확인할 수 있도록 구현하였다.

이미지 모달리티가 사용된 경우에는 attention이 높은 patch image도 별도로 저장한다.

```
if args.use_image and images is not None:
    k = min(args.topk_patches, n_spots)
    topk = torch.topk(mil_attn, k=k).indices.detach().cpu().tolist()

    patch_dir = os.path.join(out_dir, "top_patches")
    os.makedirs(patch_dir, exist_ok=True)

    for rank, i in enumerate(topk, start=1):
        fn = f"rank{rank:02d}_idx{i}"
        if barcodes is not None:
            fn += f"_bc{barcodes[i]}"
        if patch_indices is not None:
            fn += f"_pidx{int(patch_indices[i])}"
        fn += ".png"
        save_patch_image(images[i], os.path.join(patch_dir, fn))
```

본 코드는 MIL attention이 높은 spot에 대응되는 이미지 patch를 *top_patches* 폴더에 저장한다. 이를 통해 모델이 중요하게 본 조직 patch를 직접 확인할 수 있으나, 병리학 전문가와의 협업 없이는 유의미한 정보값을 도출할 수 없으리라는 판단 하에 졸업프로젝트 보고서 및 발표에서는 생략된 부분이다.

4) Gene Importance 출력

모델 내부의 ST encoder는 spatial token을 query로 사용하여 gene token들에 대한 attention을 계산한다.

```
attn = torch.softmax(
    torch.matmul(q, k.transpose(-2, -1)) / (self.embed_dim ** 0.5),
    dim=-1
)

pooled = torch.matmul(attn, v).squeeze(1)
pooled = self.out_proj(pooled)
```

```
if return_gene_attn:
    gene_attn = attn.squeeze(1)
    return pooled, gene_attn, gene_indices
else:
    return pooled
```

위 코드는 각 spot에서 spatial context를 기준으로 어떤 gene token이 ST 임베딩 생성에 더 크게 기여했는지를 계산한다. *Return_gene_attn=True*인 경우 gene attention과 해당 gene index를 함께 반환하여 후처리 단계에서 gene importance를 계산할 수 있게끔 하였다.

후처리에서는 gene attention과 MIL attention을 결합하여 WSI-level gene importance를 계산한다.

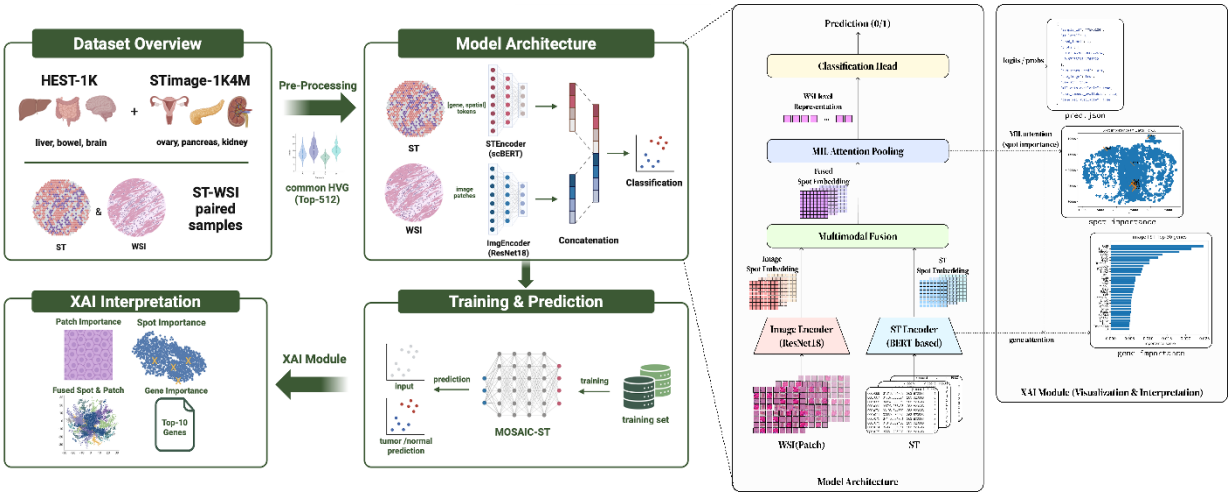
```
def aggregate_top_genes(gene_attn, gene_indices, mil_attn, gene_order, topk=30):
    N, G = gene_attn.shape
    mil_w = mil_attn.view(N, 1)

    contrib = (gene_attn * mil_w).detach().cpu().numpy()
    gidx = gene_indices.detach().cpu().numpy().astype(np.int64)

    scores = {}
    for i in range(N):
        for j in range(G):
            gid = int(gidx[i, j])
            scores[gid] = scores.get(gid, 0.0) + float(contrib[i, j])

    items = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:topk]

    out = []
    for gid, sc in items:
        gname = gene_order[gid] if (0 <= gid < len(gene_order)) else f"gene_{gid}"
        out.append((gname, sc))

    return out
```

```
if args.use_st and (gene_attn is not None) and (gene_indices is not None) and (len(gene_order) > 0):
    if gene_attn.dim() == 3:
        gene_attn2 = gene_attn.squeeze(1)
    else:
        gene_attn2 = gene_attn
```

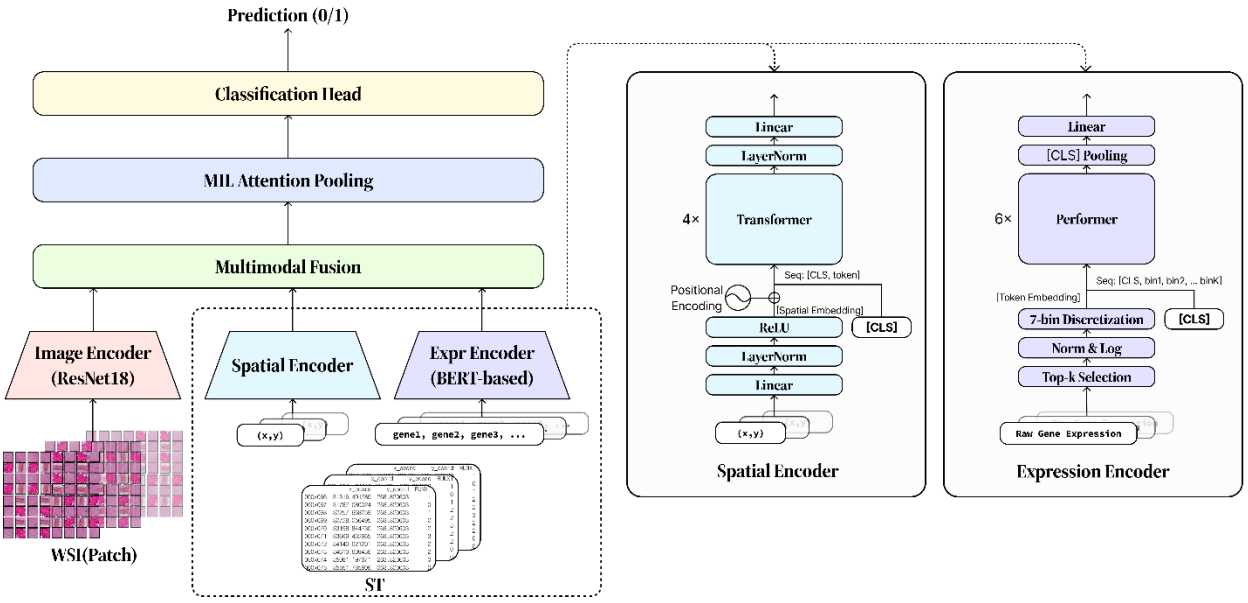
```
    top_genes = aggregate_top_genes(
        gene_attn=gene_attn2,
        gene_indices=gene_indices,
        mil_attn=mil_attn.detach().cpu(),
        gene_order=gene_order,
        topk=args.topk_genes
    )

    with open(os.path.join(out_dir, "top_genes.csv"), "w") as f:
        f.write("rank,gene,score₩n")
        for r, (g, sc) in enumerate(top_genes, start=1):
            f.write(f"{r},{g},{sc:.6f}₩n")
```

본 코드는 spot 내부의 gene attention에 spot-level MIL attention을 곱해, prediction에 중요한 spot에서 중요하게 사용된 gene이 더 높은 score를 갖도록 집계하는 방식이다. 최종적으로 score가 높은 gene을 정렬하여 *top_genes.csv*로 저장하며, ST 모달리티가 사용된 경우에만 gene importance를 계산하도록 구현하였다.

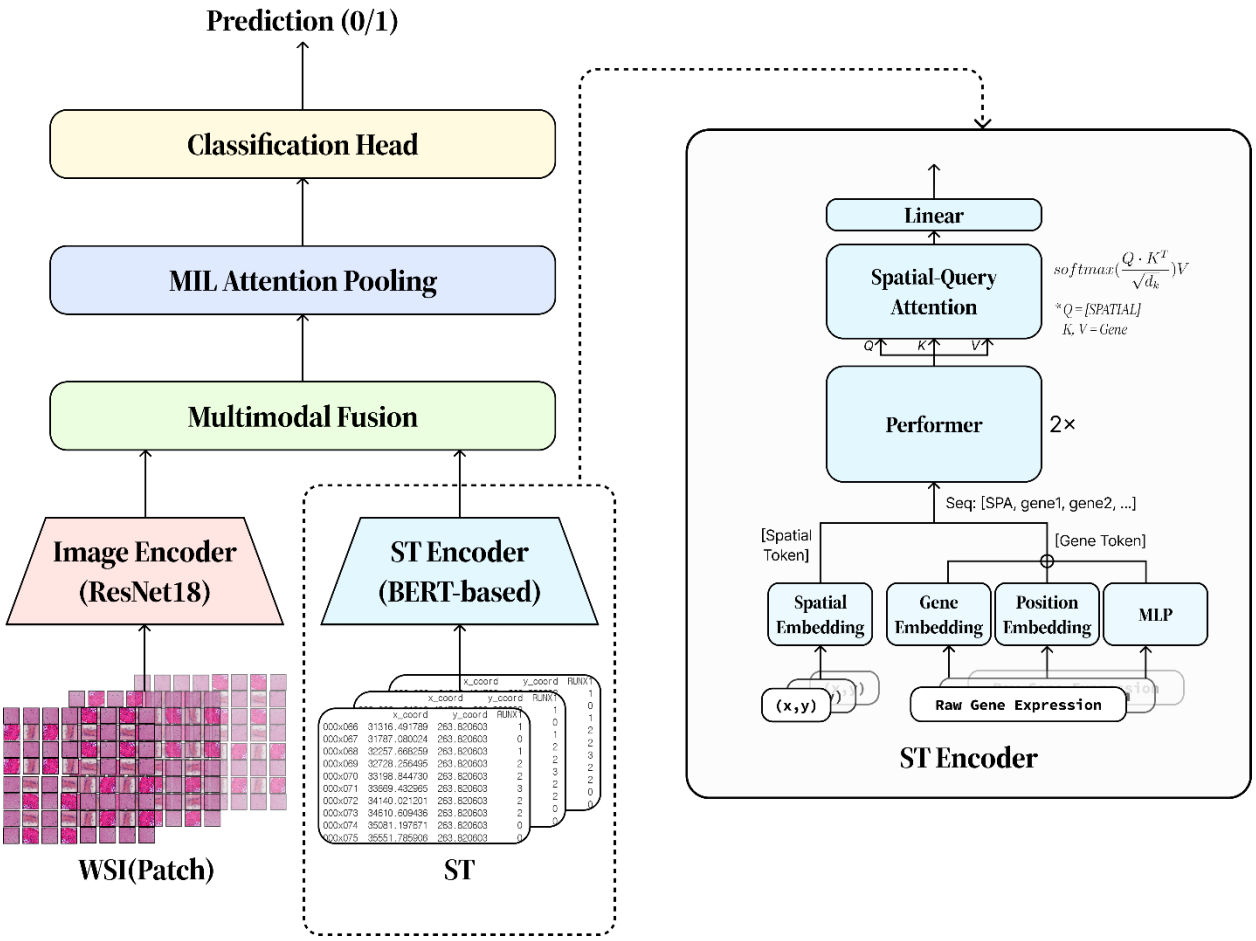
4. Appendix



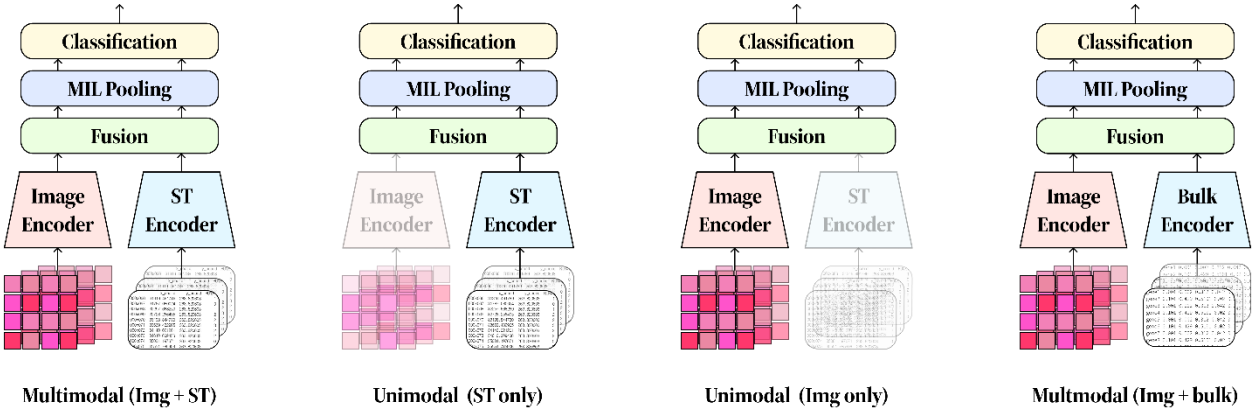
- MOSAIC-ST의 전체 파이프라인(좌측)과 모델 구조 overview(우측). 데이터셋 → 전처리 → 모델 입력 → 모델 학습 → 설명가능성 확보의 단계를 거침.



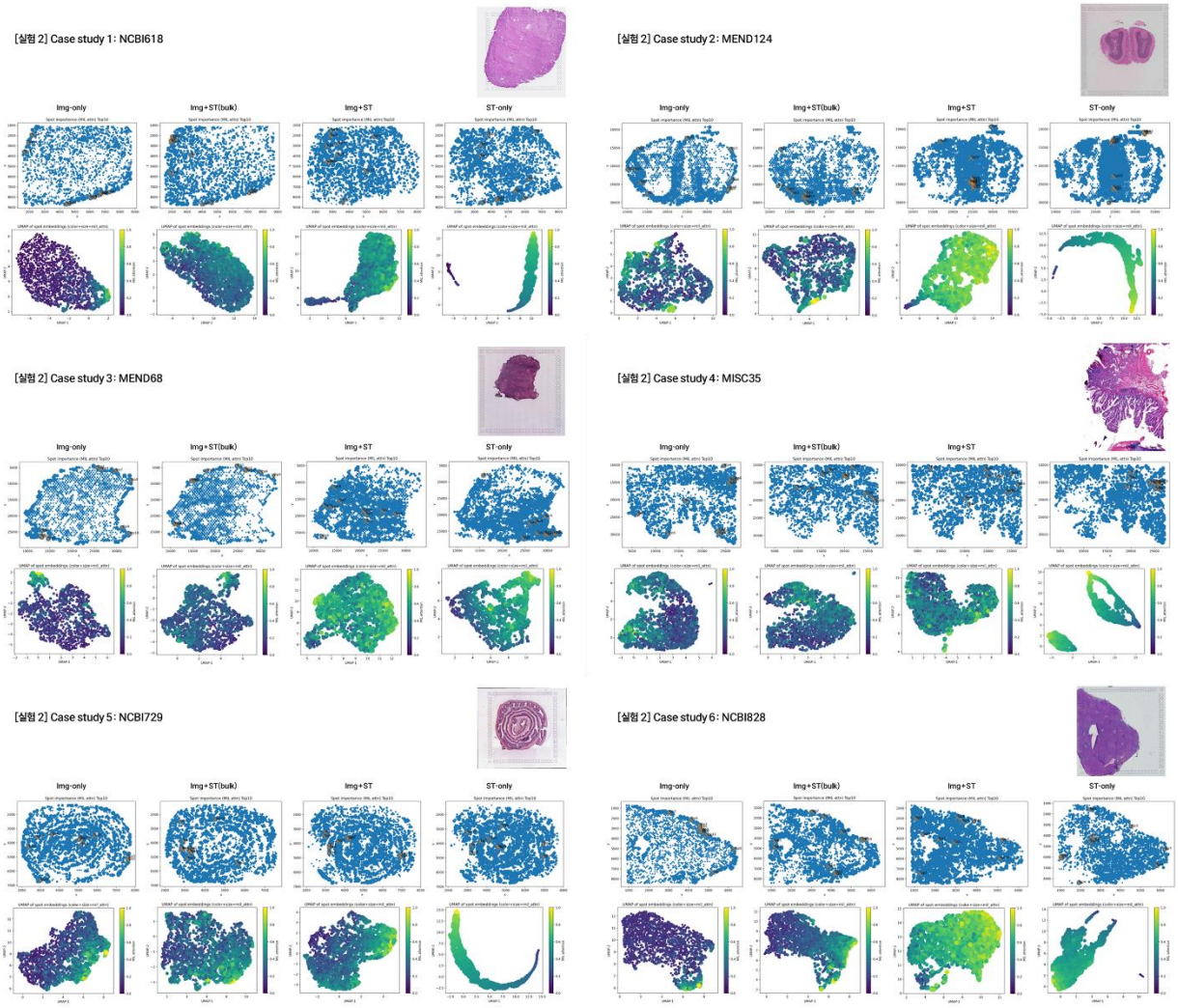
- 인코더 구조 비교 실험(실험 1)의 Ver1 모델 구조도. 좌표 데이터와 유전자 발현 데이터가 완전히 분리된 상태로 representation을 각각 생성한 뒤, 이후 fuse하는 late fuson 형태.



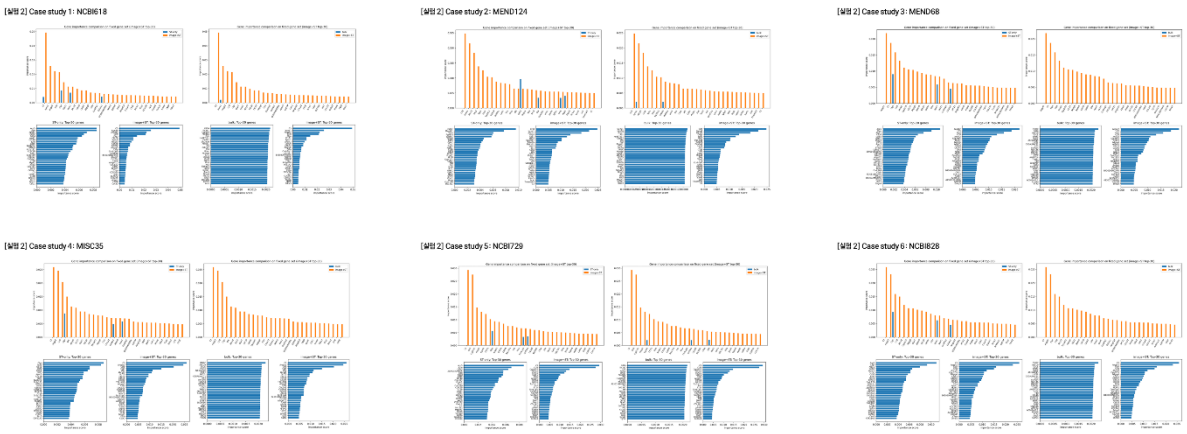
- 인코더 구조 비교 실험(실험 1)의 Ver2 모델 구조도. 좌표 데이터와 유전자 발현 데이터가 통합된 원시 ST 데이터를 단일 인코더에 입력받으며, 좌표를 query로 하는 spatial-query attention으로 representation을 생성하는 early fusion 형태.



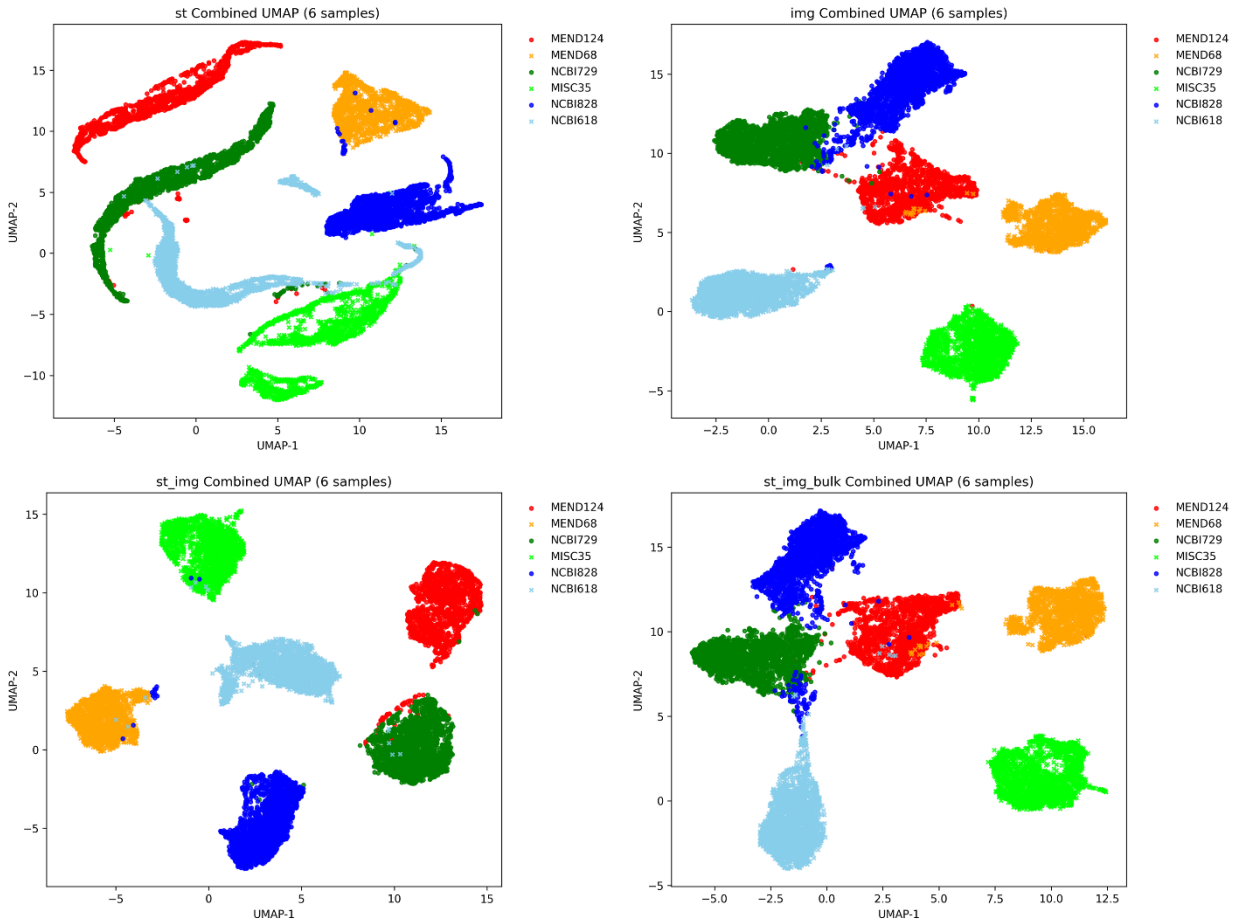
- 모달리티 비교 실험(실험 2)의 실험 세팅 구조도.



- 6개 test set에 대해 모달리티별 spot importance(upper row scatter plot)과 임베딩 UMAP(lower row UMAP)을 출력한 결과.



- 6개 test set에 대해 ST-only와 full multimodal(left), bulk multimodal과 full multimodal(right)을 비교한 결과.



- 6개 test set에 대해, 모달리티별 샘플 UMAP들을 공통 space에 proj하여 clustering quality를 비교한 결과.